

Técnicas de búsqueda y heurísticas

Diplomado en Inteligencia Artificial

Junio 2025

Profesor: Ricardo Contreras A.

¿Quién soy yo?

- Ricardo Contreras A. (M.Sc. Computer Science)
- Áreas de desarrollo: Vida Artificial, Ingeniería de Software, Teoría de Computación
- E-mail: ricardo.contreras@edu.uai.cl
 - Alternativo (rcontrer@udec.cl)

Acá estuve en mi postgrado



En esta ciudad...



Mi área de interés (hoy)



Sistemas bioinspirados



Tópicos del curso

- Algoritmos de Búsqueda
 - Estrategias de búsqueda en árbol, Juegos con adversarios, [Constraint Programming](#)
- Búsqueda informada y con oponentes
 - A*, Minimax, Optimalidad de Bellman
- Búsqueda no parcial
 - Búsqueda de Montecarlo, Aprendizaje pro-refuerzo, Algoritmos basados en poblaciones, Algoritmos adaptativos y aprendizaje
- Metaheurísticas
 - Métodos de optimización, Simulated Annealing, Tabu Search

Bibliografía Básica

- Russel, Stuart & Norvig, Peter
 - Artificial Intelligence. A modern Approach. Pearson Series in AI (2021)
- Gorriz, Juan Manuel; Contreras, Ricardo; Pinninghoff, María Angélica; et. al.
 - Artificial intelligence within the interplay between natural and artificial computation: Advances in data science, trends and applications. Neurocomputing 410, 237-270, (2020)
- Gorriz, Juan Manuel; Contreras, Ricardo; Pinninghoff, María Angélica; et. al.
 - Computational approaches to Explainable Artificial Intelligence: Advances in theory, applications and trends. Inf. Fusion 100: 101945 (2023)

Las evaluaciones

- Un trabajo grupal
 - (grupos: no menos de 3 personas, no más de 5)
- Un control (lunes 7 de julio)
- Cada uno de ellos con una ponderación del 50%

En esta clase

- Algoritmos de búsqueda
 - Estrategias de búsqueda en árbol
 - Juegos con adversarios
 - Constraint programming

Pregunta para iniciar la conversación

¿Qué sería para ustedes un “sistema inteligente”?

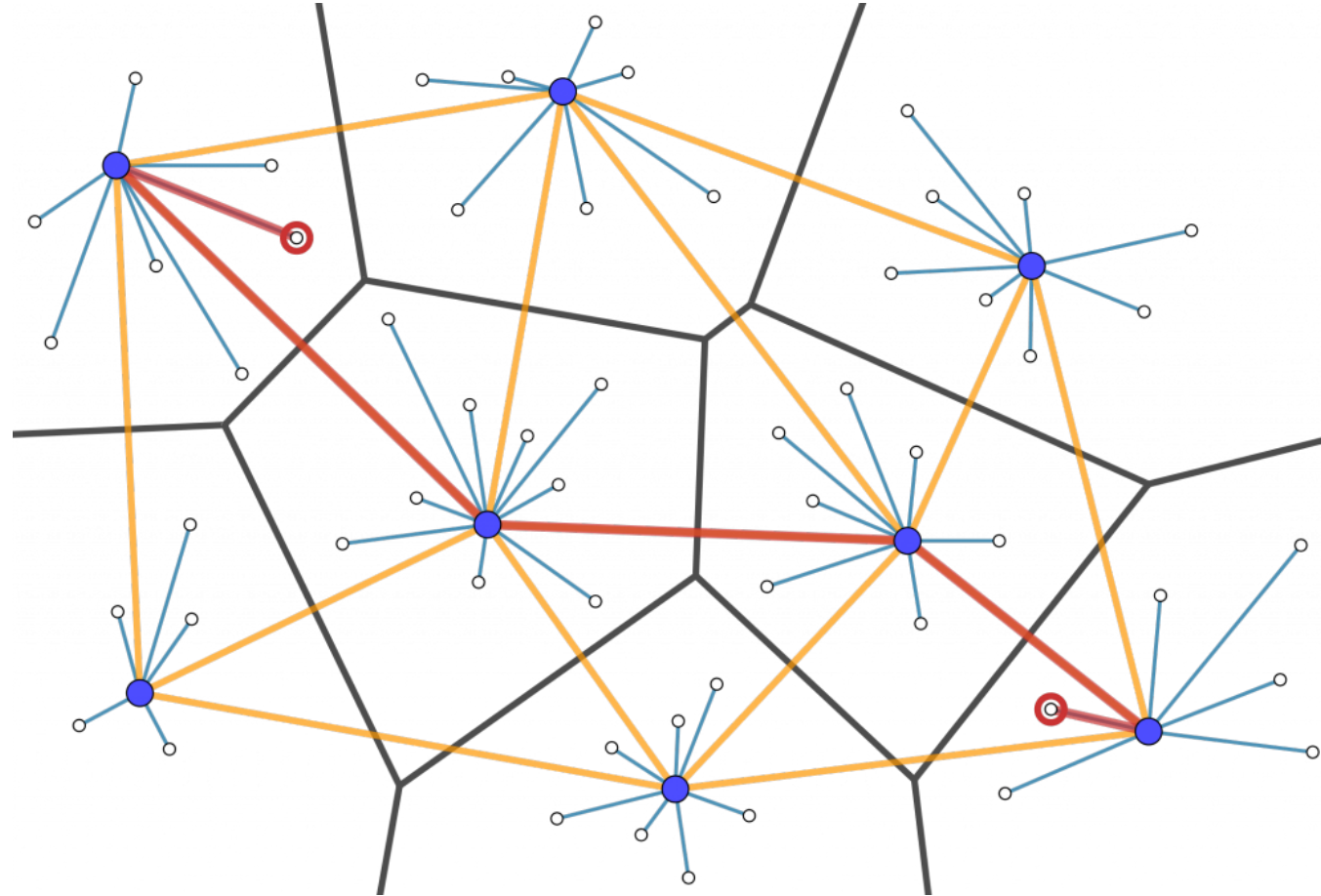
Aspectos importantes de la IA

- El uso de computadores para realizar razonamiento, reconocimiento de patrones, aprendizaje, o alguna otra forma de inferencia.
- Apunta a problemas que no obedecen a soluciones algorítmicas clásicas .
- Resolución de problemas con información inexacta o incompleta (difusa).

Aspectos importantes de la IA (cont.)

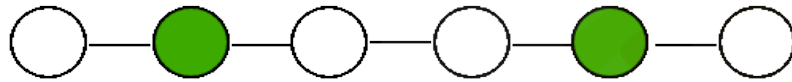
- Razonamiento sobre características cualitativas significativas de una situación.
- Genera respuestas que sin ser exactas u “óptimas” en algún sentido son suficientes.
- El uso de meta-conocimiento para el uso de estrategias complejas de solución de problemas.

Una estructura interesante

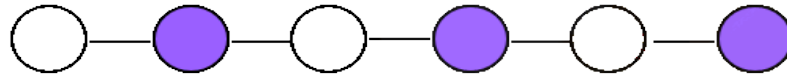


Problemas clásicos en grafos

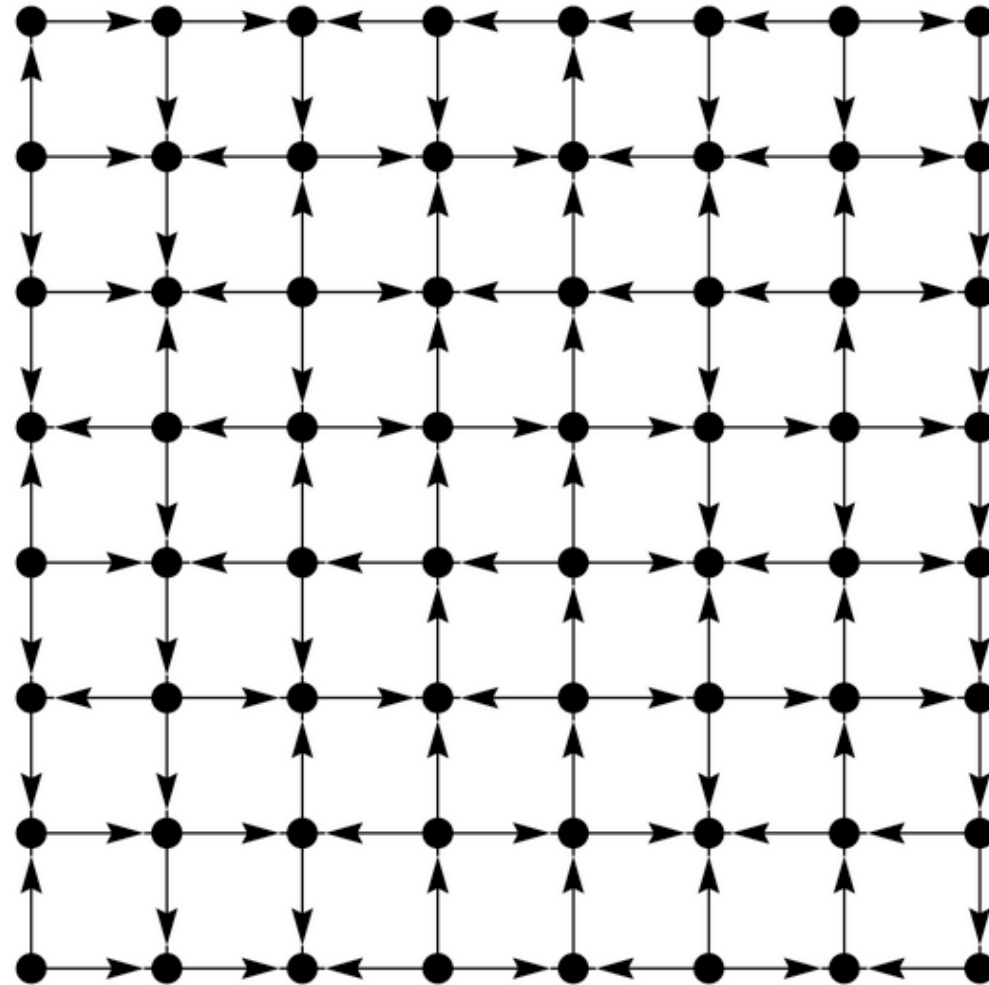
- Conjunto dominante



- Conjunto independiente



Los grafos tienen muchas aplicaciones



¿Por qué estudiar IA?

Los primeros intentos de **vuelo** consistían en máquinas que imitaban el movimiento de las alas de las aves.



¿Por qué estudiar IA?

(Cont.)

Los actuales **aviones tripulados** pueden acarrear grandes cantidades de carga a velocidades mayores que la velocidad del sonido; y no se trata de máquinas que imiten el movimiento de las alas de las aves.



Heurística

El término **heurística** viene del griego [*εὕρισκειν*](#) que significa hallar, inventar.

El pretérito perfecto de este verbo es [*eureka*](#) (*εὕρηκα*)

El término fue utilizado por [*Albert Einstein*](#) en la publicación sobre el [*efecto fotoeléctrico*](#) (1905), con el cual obtuvo el premio Nobel en Física en el año 1921 y cuyo título es: “Sobre un punto de vista **heurístico** concerniente a la producción y transformación de la [*luz*](#)” (*Über einen die Erzeugung und Verwandlung des Lichtes betreffenden heuristischen Gesichtspunkt*)

Heurísticas (en el cotidiano)

- Las heurísticas son atajos cognitivos que permiten tomar decisiones rápidas sin procesos complejos de pensamiento
- En general se basan en
 - Experiencias pasadas
 - Normas culturales
 - Expectativas sociales
- Pueden ser útiles para navegar en la vida diaria, pero también pueden desviarnos si confiamos ciegamente en ellas

No son heurísticas...



Un caso futbolero (ocurrido hace tiempo... en otro país)



¿Y las metaheurísticas?

- Una heurística es una aproximación que encuentra rápidamente una solución factible, no necesariamente optimal, para un problema de optimización complejo. Generalmente una heurística está orientada a un problema específico.
- Una metaheurística es una estrategia de alto nivel, independiente del problema, diseñada para guiar una búsqueda, y que sirve para resolver un conjunto amplio de problemas.

Búsqueda Heurística

- Heurística - una “rule of thumb” (a veces lo podemos traducir como regla de oro). Es un principio o una guía precisa basada en la práctica y no en la teoría, usada para conducir un proceso (en nuestro caso, de búsqueda).
- A menudo, se entiende como algo que se aprende experimentalmente y que se recuerda cuando se necesita.

Búsqueda Heurística (Cont.)

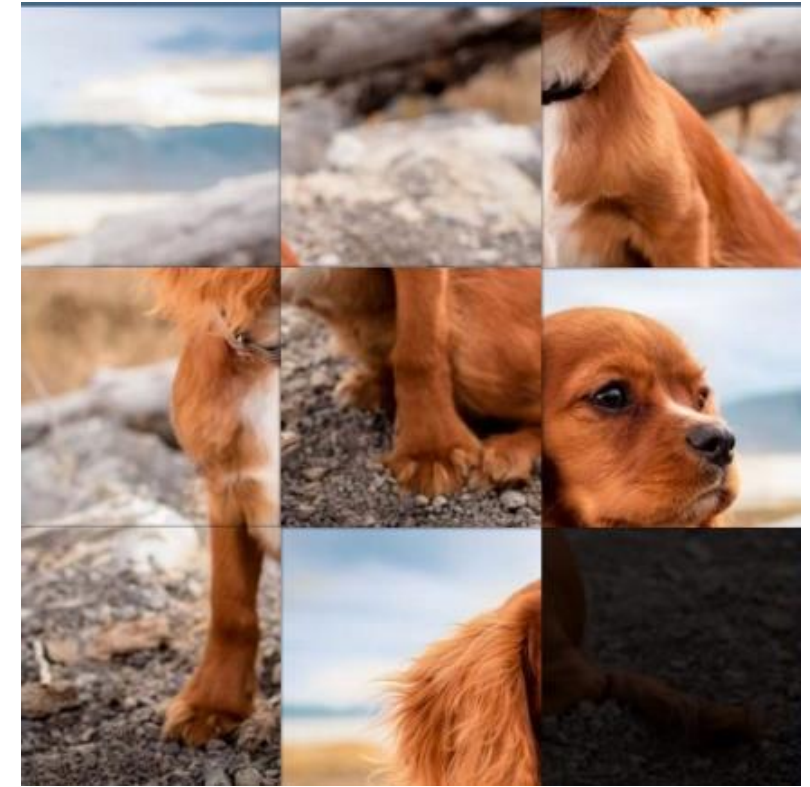
- Función heurística – es una función que se aplica a un estado, en un espacio de búsqueda, que nos indica cuán cercano al éxito es ese estado
 - Los métodos de búsqueda heurística se conocen como “métodos débiles” a causa de su generalidad y porque no aplican una gran cantidad de conocimiento
 - Los métodos no son específicos a un dominio o para un problema, solo la función heurística es específica para el problema

Búsqueda Heurística

- A partir de un espacio de búsqueda, un estado actual y un estado objetivo:
 - Generar todos los estados sucesores y evaluar cada uno de ellos con la función heurística
 - Seleccionar el movimiento que nos conduzca al mejor valor heurístico

Ejemplo de Función Heurística

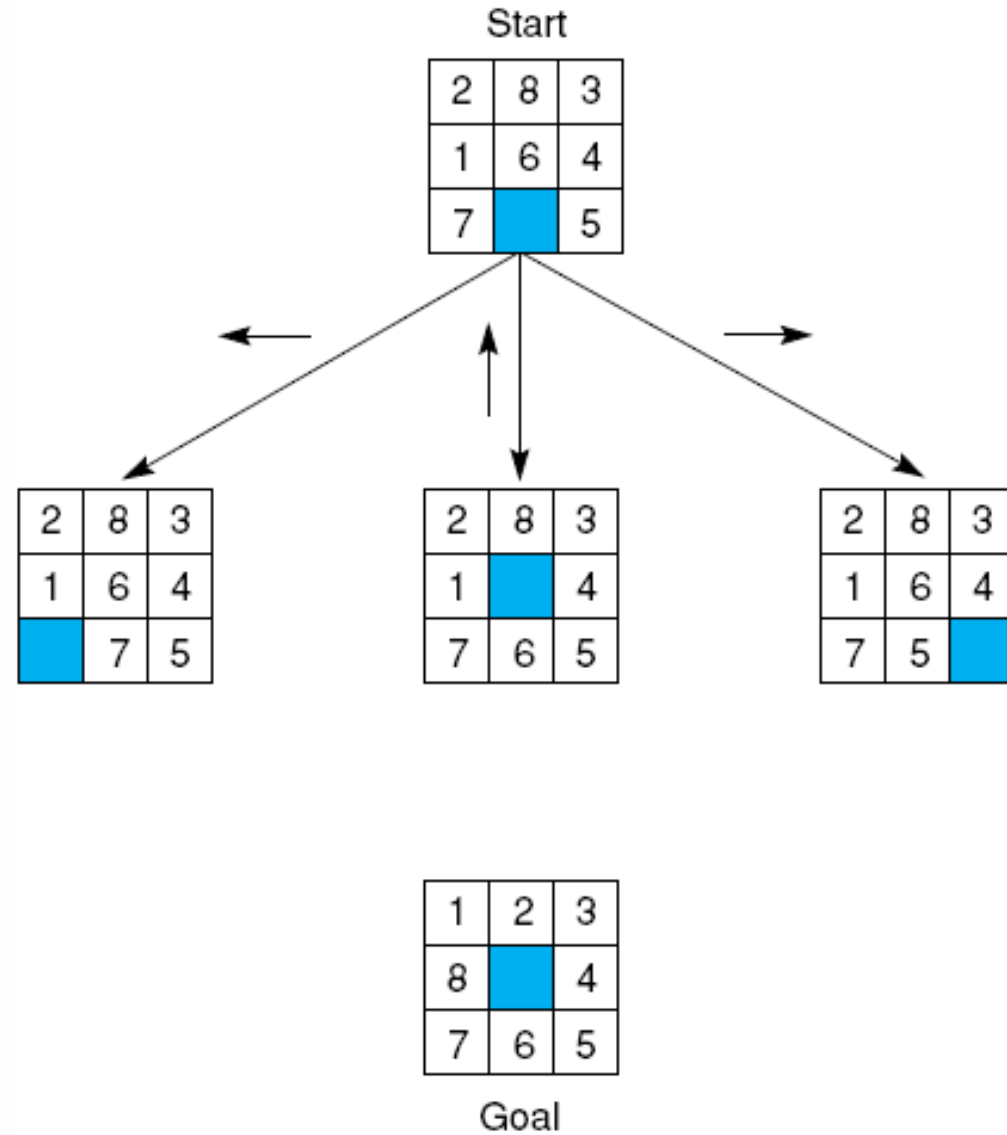
- Heurística simple para el puzzle-8:
 - Agregar 1 punto por cada pieza en la posición correcta
 - Restar 1 punto por cada pieza en la posición incorrecta
- Heurística mejorada para el puzzle-8:
 - Agregar 1 punto por cada pieza en la posición correcta
 - Restar 1 punto por cada movimiento necesario para llevar una pieza a la posición correcta



Ejemplo de Función Heurística

- La primera heurística solamente considera la posición local de la pieza
 - No considera factores como grupos de piezas en la posición correcta
 - Podríamos diferenciar entre estos dos tipos de heurísticas, denominándolas *local* vs *global*

Ejemplo: Puzzle-8



Resolución del puzzle-8 vía heurísticas

Una mirada rápida

Heurísticas

- Admisibles

- Una heurística admisible nunca sobre-estima el costo de alcanzar una meta. Así, el uso de una heurística admisible siempre produce una solución óptima

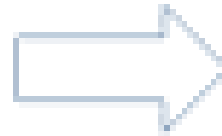
- No admisibles

- Una heurística no admisible puede sobre-estimar el costo de alcanzar una meta. Así el uso de una heurística no admisible puede resultar (o no) en una solución óptima.
- Sin embargo, la ventaja de esta opción es que a veces una heurística no admisible expande **muchos menos nodos**.
- Consecuencia de ello es que a veces el costo total (igual al costo de la búsqueda más el costo del recorrido) puede ser menor que el costo de usar una heurística admisible.

Heurísticas admisibles para el puzzle-8

Estado inicial

7	2	4
5	*	6
8	3	1



Estado Final

*	1	2
3	4	5
6	7	8

Heurística 1

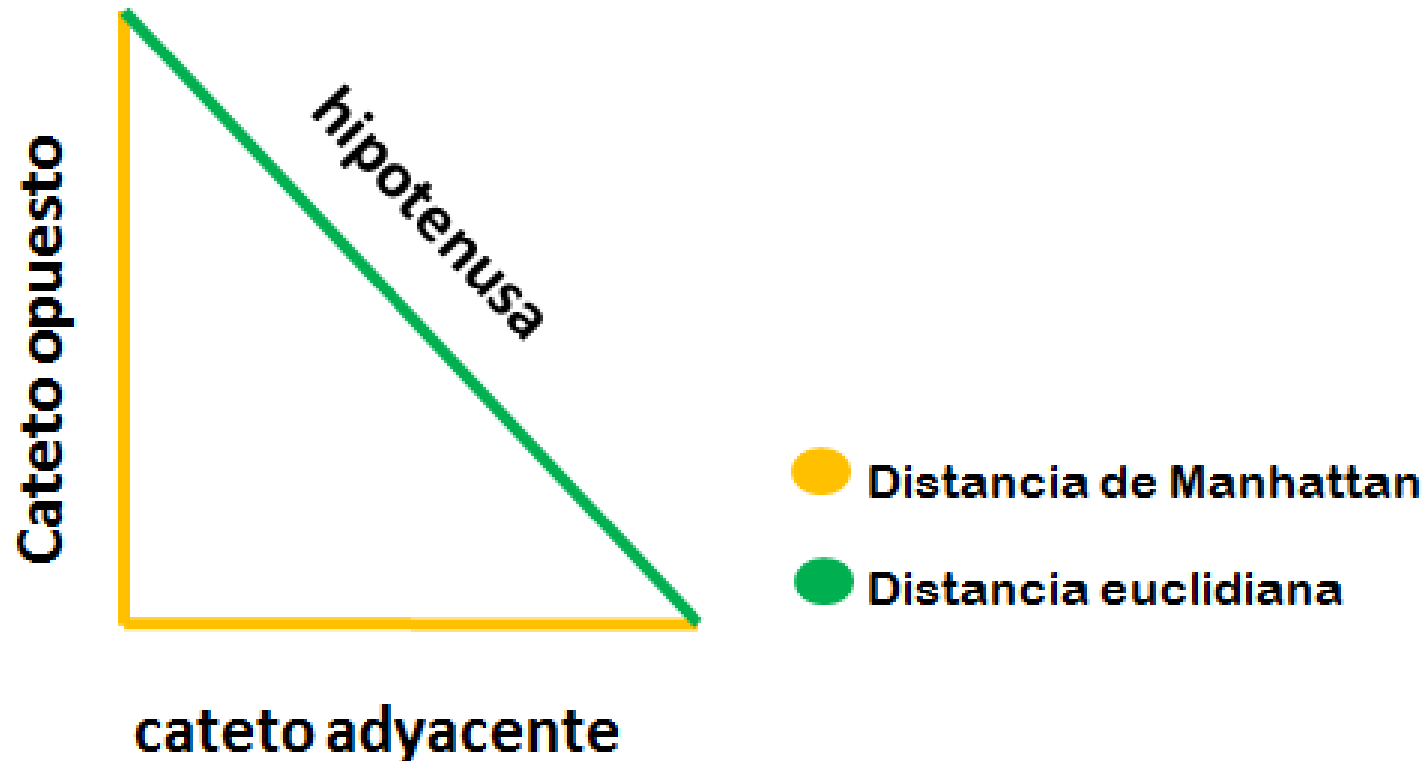
- h_1 : Número de piezas fuera de lugar
- En la figura anterior, todas las piezas están fuera de su posición final, por lo que para este estado, $h_1 = 8$
- h_1 es una heurística admisible, ya que es claro que cada pieza que está fuera de su posición final debe ser movida al menos una vez

Heurística 2

- h2: Suma de las distancias euclideanas de las piezas a sus posiciones finales
- En la figura, todas las piezas están fuera de posición, por lo que en ese estado el valor de h2 se calcula de la manera siguiente:
- $h2 = \sqrt{5} + 1 + \sqrt{2} + \sqrt{2} + 2 + \sqrt{5} + \sqrt{5} + 2 = 14.53$.
- h2 es una heurística admisible, ya que, en cada movimiento, una pieza sólo puede moverse para acercarse a su meta un paso; y la distancia euclideana nunca es mayor al número de pasos requeridos para mover una pieza a su posición final

Heurística 3

- h3: Suma de las distancias Manhattan de las piezas a sus posiciones finales



Heurística 3

- $h3$: Suma de las distancias Manhattan de las piezas a sus posiciones finales
- Como ya sabemos, en el estado inicial, todas las piezas están (para este caso) fuera de posición, por lo tanto para esta situación, $h3$ es
- $h3 = 3 + 1 + 2 + 2 + 2 + 3 + 3 + 2 = 18$
- $h3$ es una heurística admisible, ya que en cada movimiento, una pieza puede acercarse a su posición final un solo paso

Heurística 4

- h4: Número de piezas fuera de su fila (final) + número de piezas fuera de su columna (final)
- En este caso, todas las piezas tendrán un valor mayor a cero
- $h4 = 5 \text{ (fuera de fila)} + 8 \text{ (fuera de columna)} = 13.$
- h4 es una heurística admisible, ya que cada pieza que está fuera de una columna o de una fila deberá ser movida al menos una vez, y cada pieza que esté fuera de la columna y de la fila final, deberá ser movida al menos dos veces

Algunas heurísticas admisibles adicionales

- n-Max Swap

Suponga que se puede intercambiar cualquier pieza con el “*espacio*”. Use el costo de la solución optimal como una heurística para este problema

- n-Swap

Represente el “*espacio*” como una pieza y suponga que puede intercambiar dos piezas cualesquiera. Use el costo de la solución optimal para este problema como una heurística para el puzzle-8

Heurísticas de esta clase, implican la realización de búsquedas de forma relajada para un problema (es un método para inventar funciones heurísticas admisibles)

Concepto de estados y
árboles de búsqueda

Estados

- Estado: conjunto de variables/valores
 - Estado inicial
 - Estado actual
 - Estado objetivo
- Espacio de estados: estados posibles
 - Combinación de valores de variables

Búsqueda

- Componentes de un sistema de búsqueda
- Definición
- Razonamiento

Búsqueda

- Componentes
 - Base de Datos:
 - Estado actual y objetivo
 - Conjunto de operadores:
 - Transforman un estado en otro
 - Estrategia de control:
 - Decide qué hacer a continuación

Búsqueda

- Definición
 - Una secuencia de operadores, finita, que transforman el estado inicial en el estado objetivo
- Razonamiento
 - Forward
 - A partir del estado **inicial** encuentra el estado **objetivo**
 - Backward
 - A partir del estado **objetivo** encuentra el estado **inicial**

Búsqueda

- Espacio de estados
 - Un operador produce un nuevo estado
- Reducción de problema
 - Un operador produce un conjunto de subproblemas que deben ser resueltos

Ejemplo



- Una persona va a buscar 2 litros de agua a un pozo cercano.
- Para ello lleva dos baldes, uno de 5 litros y uno de 3 litros sin medidas intermedias.
- ¿Cómo puede regresar con la cantidad de agua pedida?

La especificación del problema

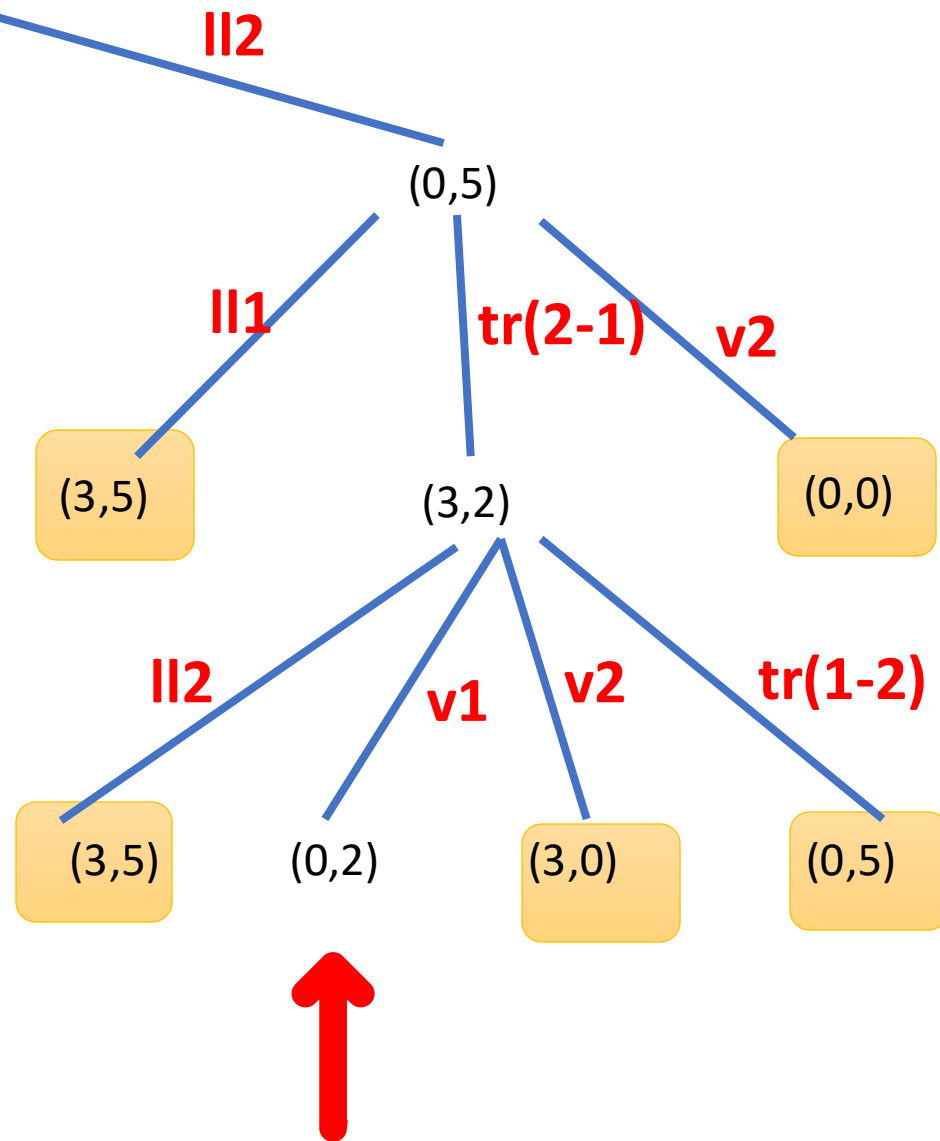
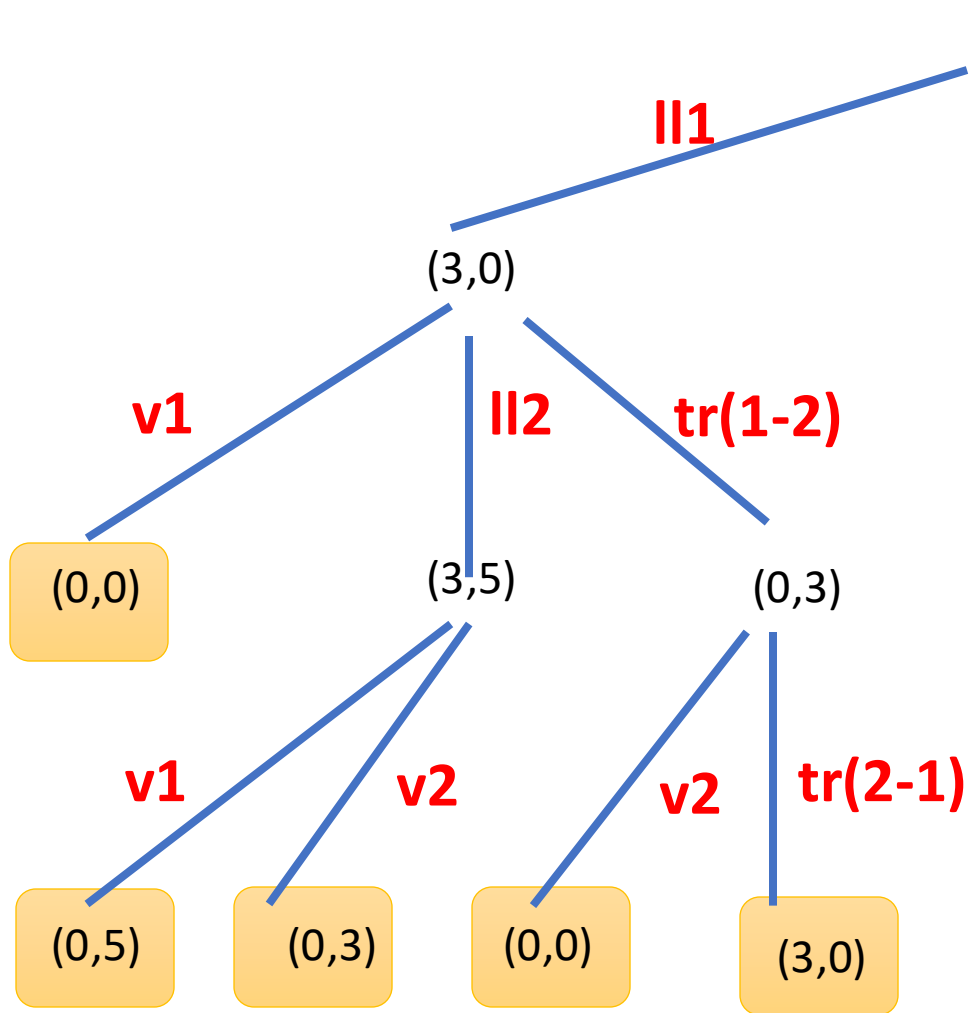
- Estados: B1, B2
- Funciones: ll1, ll2, v1, v2, tr(1-2), tr(2-1)
- Estado inicial: (0,0)
- Estados finales: (*,2), (2,*)

Admisibilidad de funciones

- $ll1$, si $B1 < 3$
- $ll2$, si $B2 < 5$
- $v1$, si $B1 > 0$
- $v2$, si $B2 > 0$
- $tr(1-2)$, si $B2 < 5$ y $B1 > 0$
- $tr(2-1)$, si $B1 < 3$ y $B2 > 0$

A partir del estado inicial: $(0,0)$

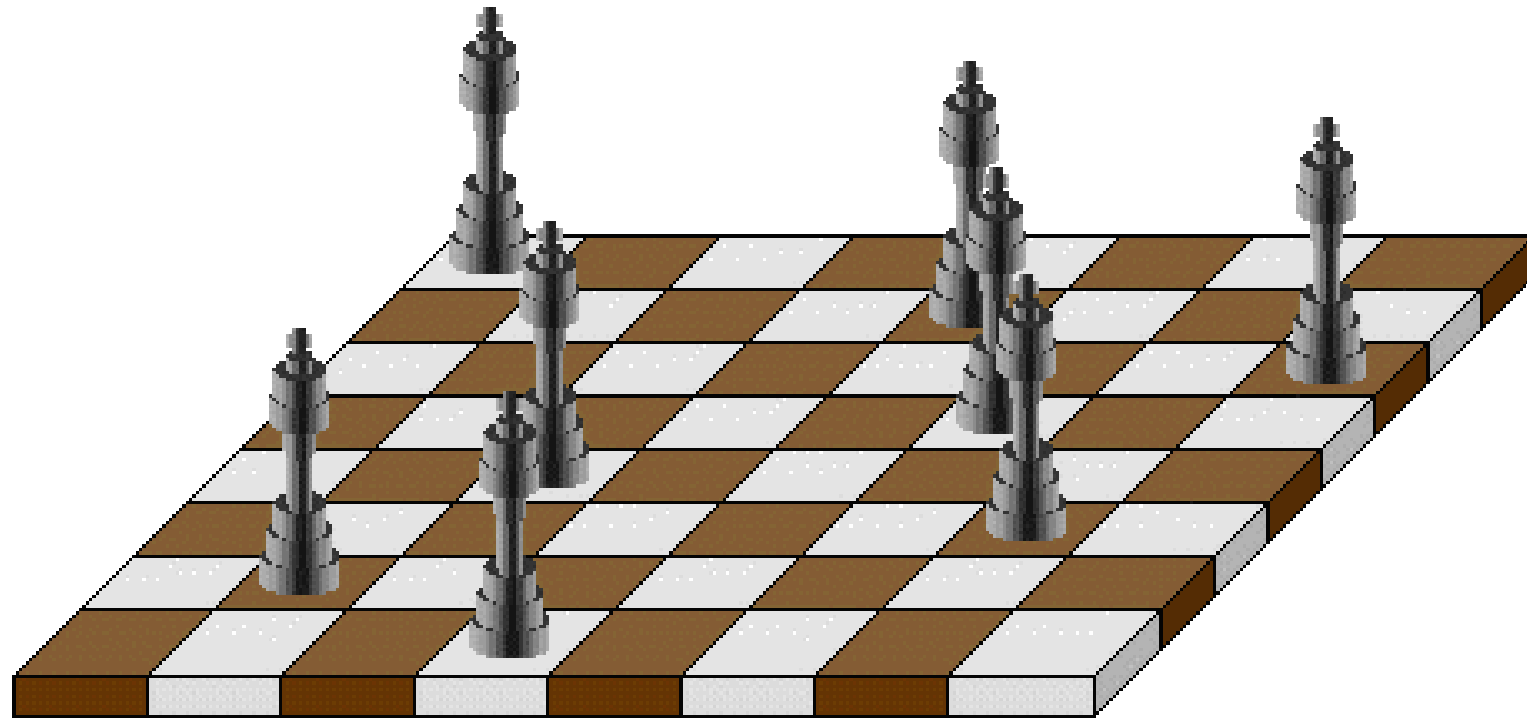
- Funciones admisibles: l_1, l_2
- Funciones no admisibles: $v_1, v_2, tr(1-2), tr(2-1)$



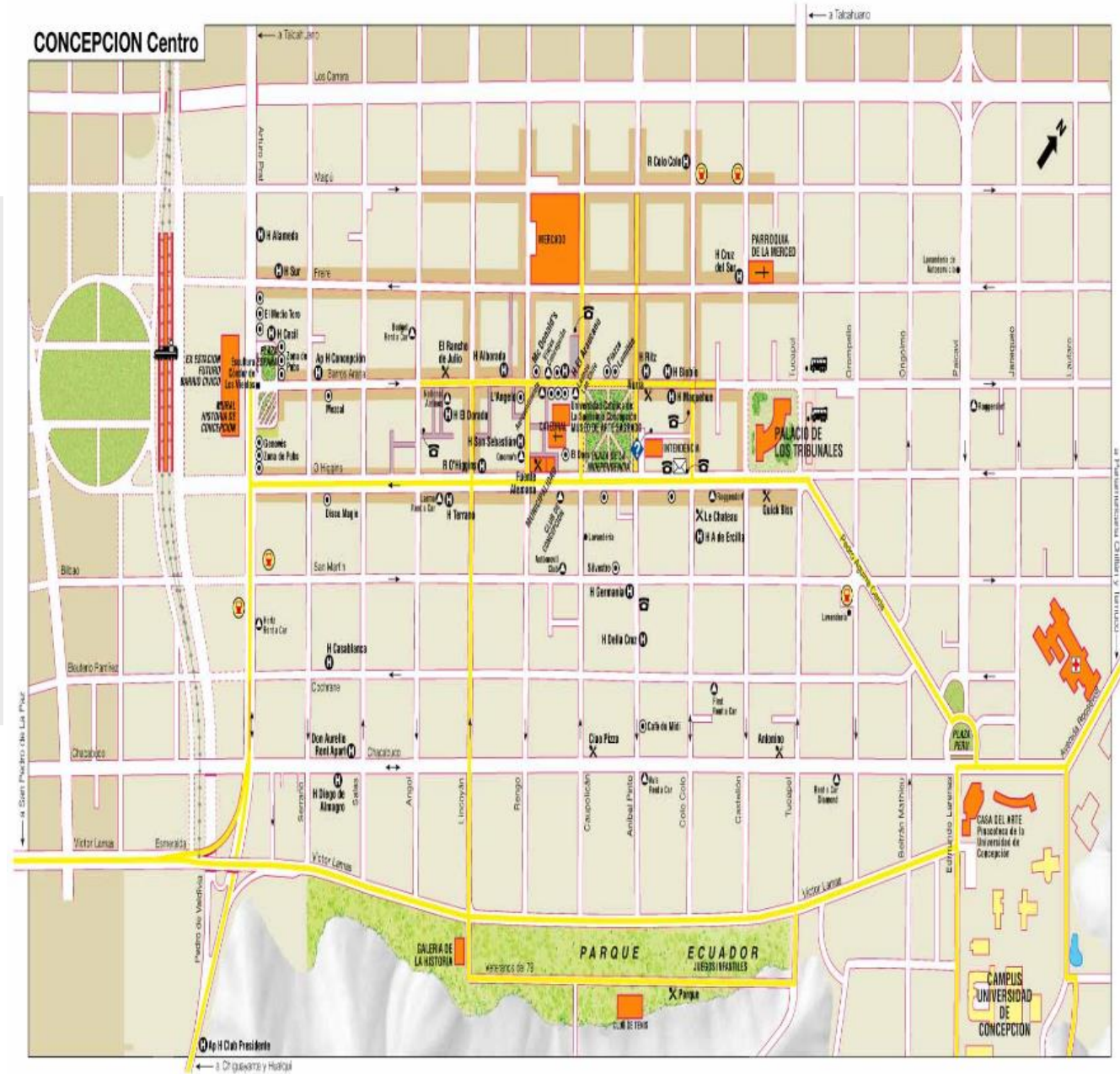
2	8	3
1	6	4
7		5

Puzzle-8

¿Es posible colocar 8 reinas en un tablero de ajedrez de forma que no se ataquen entre ellas?



Encontrar una ruta desde la Estación Central a la Universidad tal que el tiempo de conducción sea mínimo, ... tal que el kilometraje sea mínimo, ... tal que el número de semáforos sea mínimo ...



Búsqueda

El problema del Granjero, el Lobo la Cabra y el Repollo

Un granjero con su lobo, una cabra y un repollo, llegan a la orilla de un río que desean cruzar.

Hay un bote en la orilla del río, pero por supuesto sólo el granjero puede remar.

El bote puede contener sólo hasta dos objetos a la vez (incluido el remero).

Si el lobo es dejado solo con la cabra en una orilla, se la comerá; de forma similar, si la cabra es dejada sola con el repollo en una orilla, se lo comerá.

¿Cómo puede el granjero cruzar el río de manera que los cuatro “objetos” lleguen sin novedad a la otra orilla?

Este problema se remonta al siglo VIII de los escritos de Alcuin, un poeta, educador, clérigo y amigo de Carlomagno.

Este es un problema que tiene muchas variaciones

- ➡ (Granjero, Zorro, Pollo, Trigo)
- ➡ (Granjero Perro, Conejo, Lechuga)
- ➡ Y por supuesto, misioneros y caníbales

- El problema del Granjero, el Lobo, ... Puede ser resuelto fácilmente usando búsqueda. La pregunta es ¿por qué perder el tiempo con una técnica trivial para resolver problemas?
- El problema del Granjero, el Lobo, ... ¿tiene un espacio de búsqueda pequeño!
- Sin embargo, muchos problemas reales tienen espacios de búsqueda muy grandes (posiblemente infinitos). ¿Cómo buscamos en un espacio que tiene más estados que la cantidad de electrones del universo?

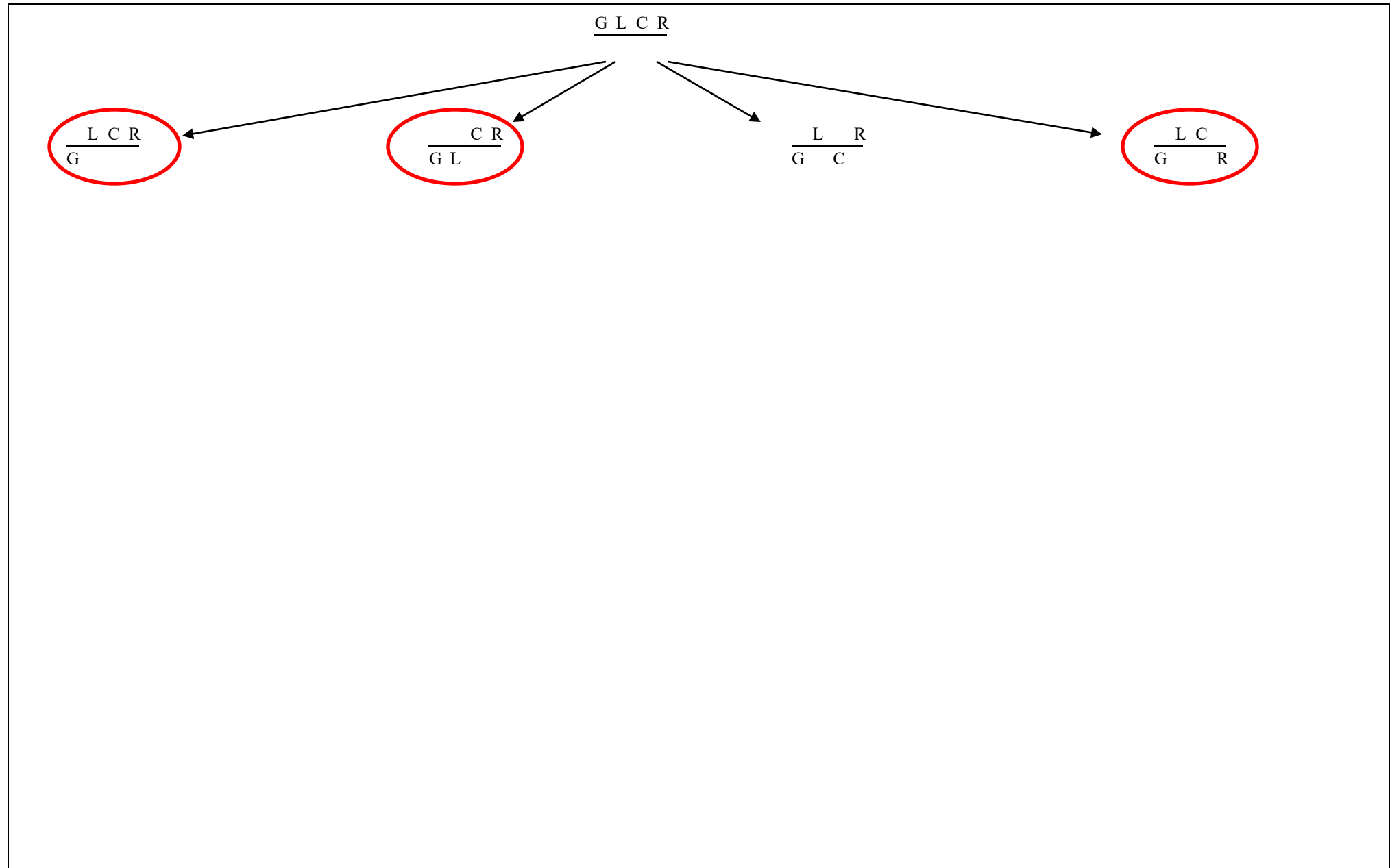
- Este problema supone que tenemos un **conocimiento completo** (siempre sabemos donde está cada cosa) y un mundo **estático** (el río no cambia, el bote es siempre el mismo, etc.).
- Sin embargo en los problemas reales no tenemos un **conocimiento completo** del estado actual del mundo, más aún, el **mundo está cambiando** de formas que no podemos predecir ni controlar.

Esto significa que todos
los objetos/persona están
al mismo lado del río

$$\underline{G \ L \ C \ R}$$

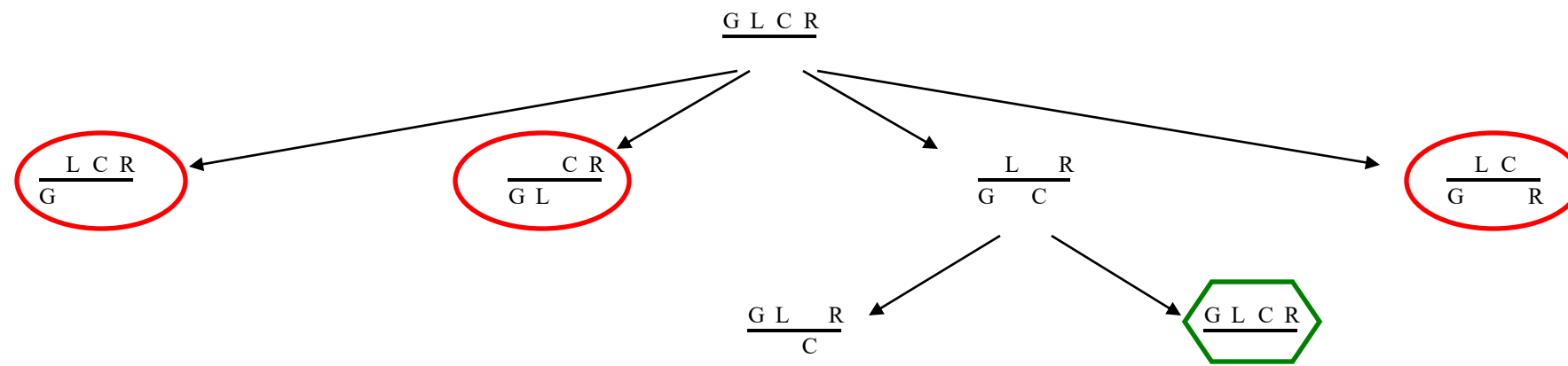
Esto significa que de
algún modo hemos
llevado al lobo al otro
lado del río

$$\begin{array}{c} G \quad C \ R \\ \hline L \end{array}$$



Arbol de Búsqueda para “Granjero, Lobo, Cabra, Repollo”

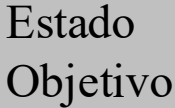
Estado Ilegal



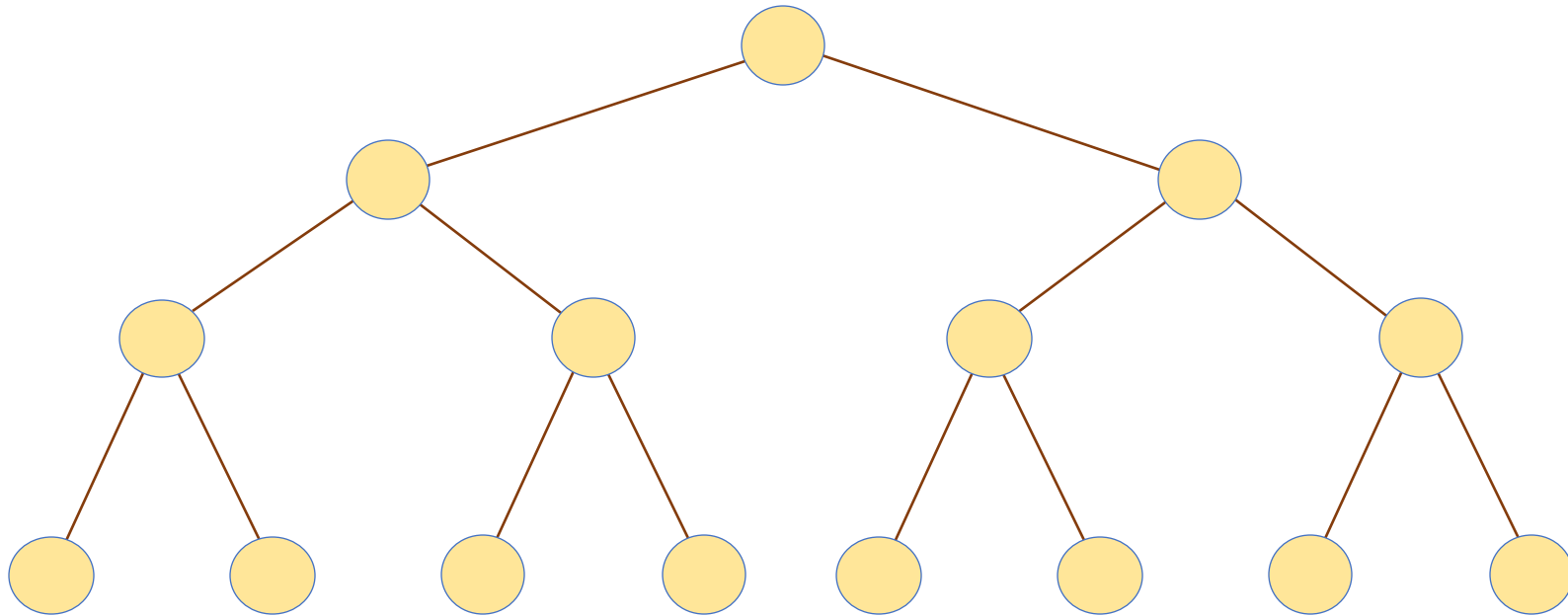
Arbol de Búsqueda para “Granjero, Lobo, Cabra, Repollo”

Estado Ilegal

Estado Repetido



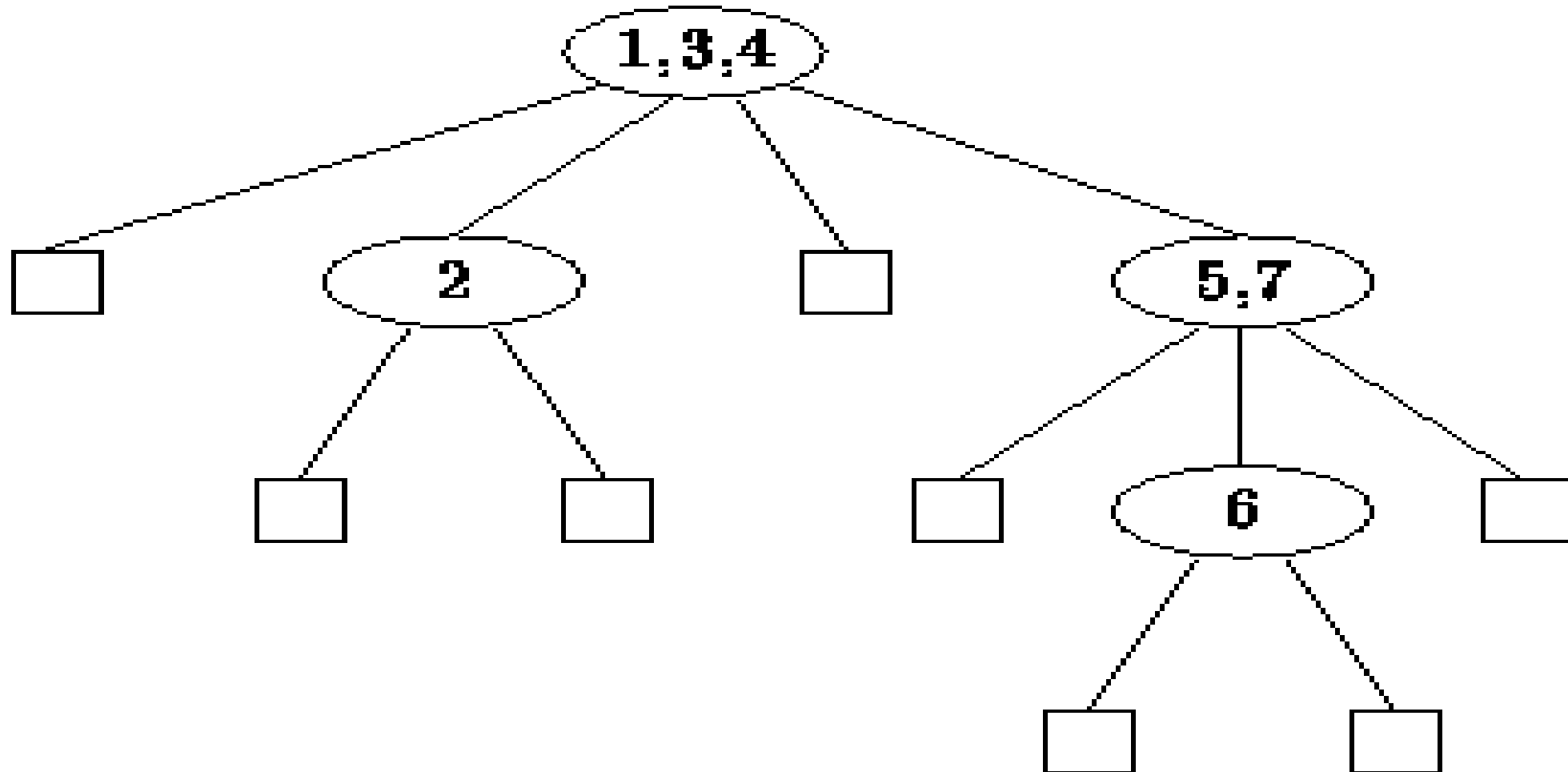
árboles



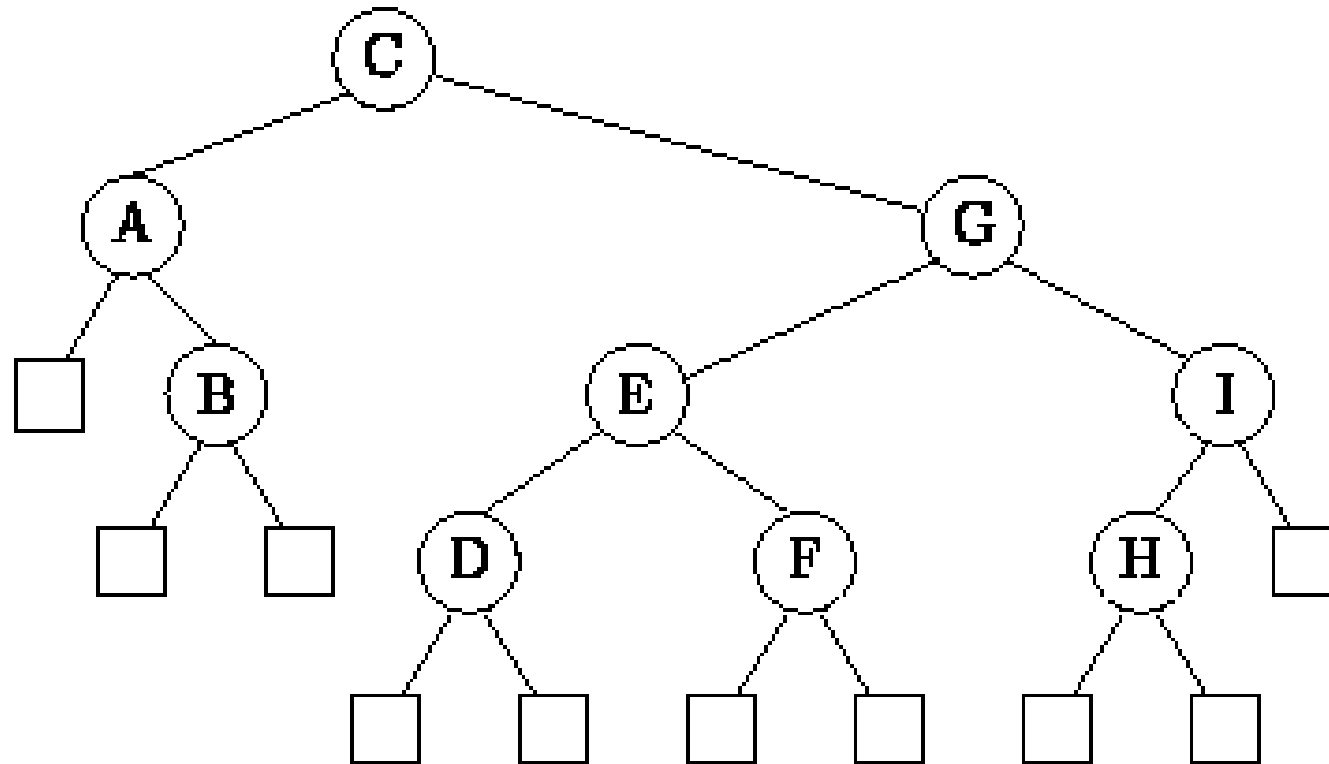
Árboles de Búsqueda

- Un árbol que permite operaciones eficientes de búsqueda, se llama *árbol de búsqueda*.
- El árbol se usa para almacenar un conjunto finito de claves obtenidas desde un conjunto totalmente ordenado de claves K .
- Cada nodo del árbol contiene una o más claves y todas las claves son únicas.
- La diferencia entre un árbol común y un árbol de búsqueda es que, en este último, las claves no aparecen en nodos arbitrarios. Existe un *criterio de ordenación de datos* que determina dónde cada clave puede aparecer en el árbol, en relación a las demás.
- Los árboles de búsqueda pueden ser:
 - n -arios (M-Way)
 - binarios.

Ejemplo de árbol de búsqueda M-Way



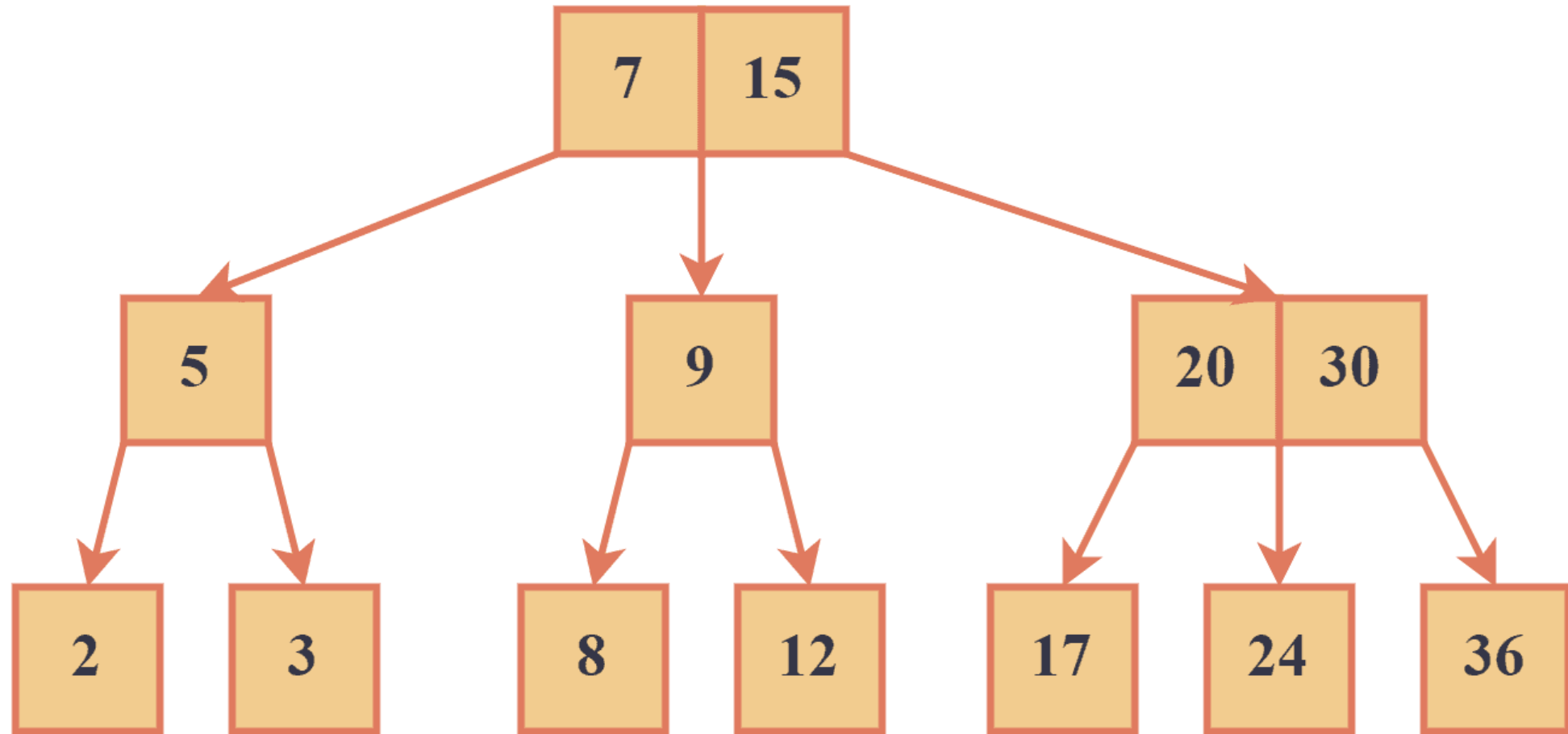
Ejemplo de árbol binario de búsqueda



Busqueda en Árboles n-arios (M-Way)

- Búsqueda del objeto x comienza en la raíz
- Si la raíz está vacía, la búsqueda falla
- Las claves en la raíz se examinan para verificar si el objeto buscado está presente
- Si el objeto no está presente, pueden darse 3 casos:
 - El objeto buscado es menor que k_1
 - la búsqueda prosigue en T_0 ;
 - El objeto buscado es mayor que k_{n-1}
 - la búsqueda prosigue en T_{n-1} ;
 - Existe un i tal que $k_i < x < k_{i+1}$
 - la búsqueda prosigue en T_i

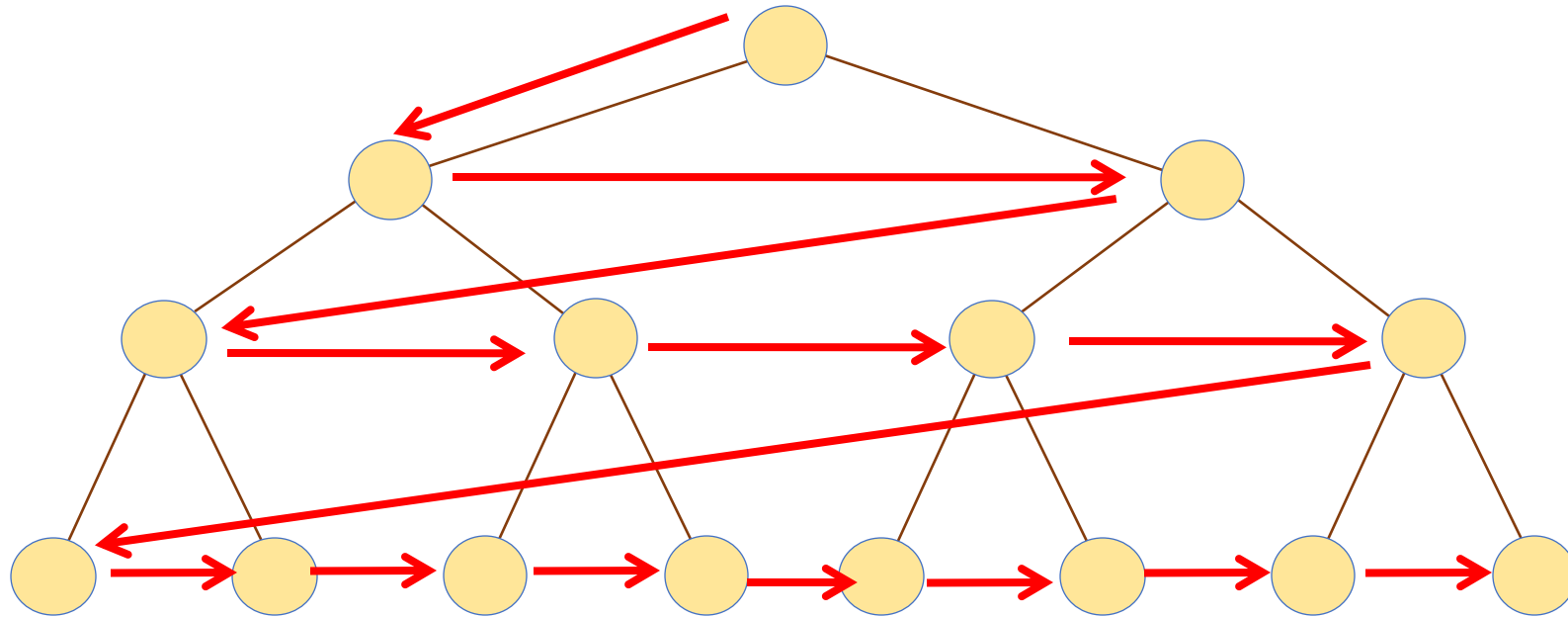
Este árbol de búsqueda es un árbol 2-3



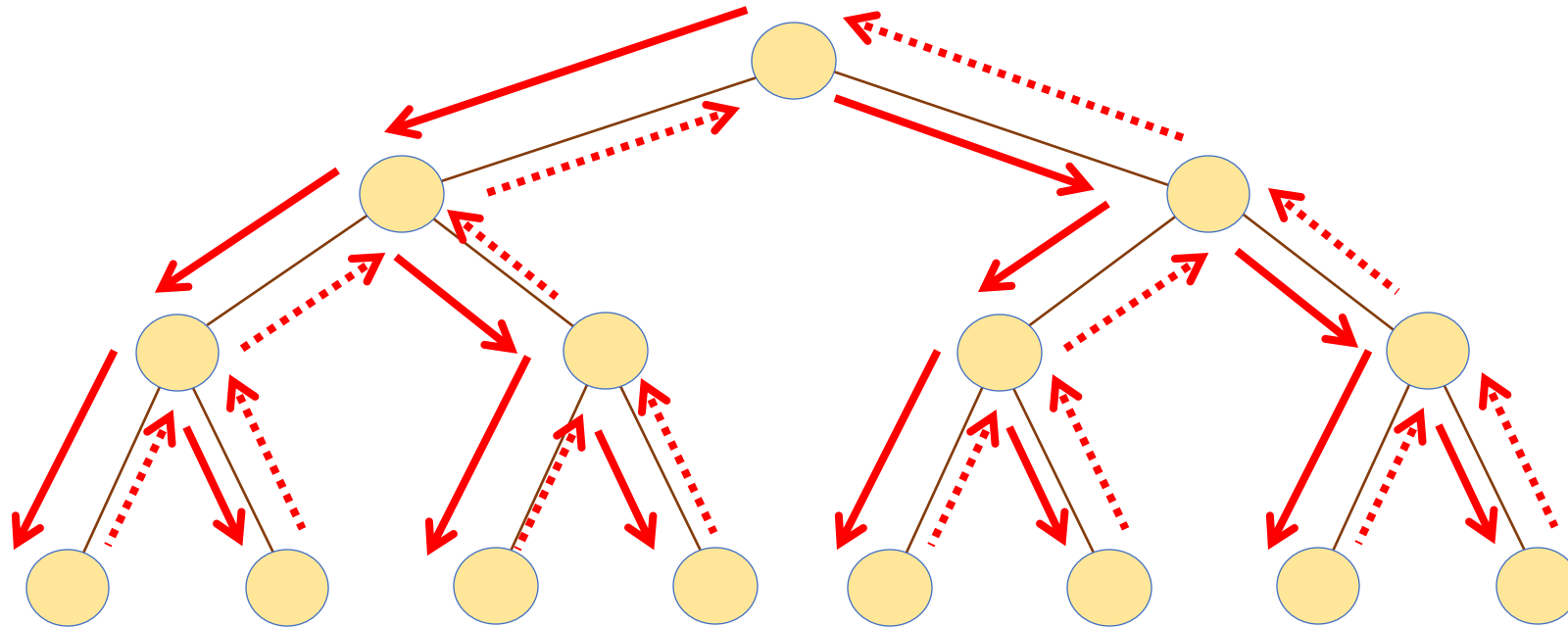
Búsqueda en Árboles Binarios

- Semejante a la búsqueda en árboles M-Way
- Cada nodo solo tiene un objeto y dos sub-árboles

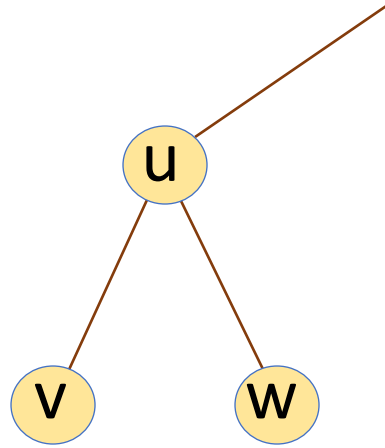
Breadth First Search (BFS) (En anchura)



Depth First Search (DFS) (en profundidad)

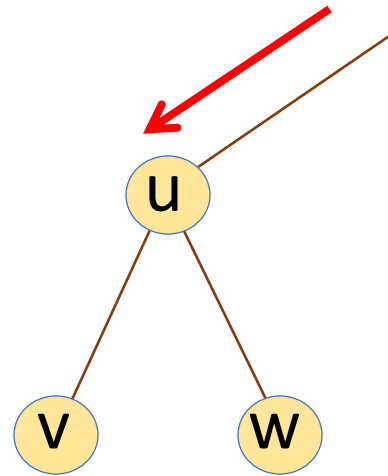


Depth-First Search (DFS)



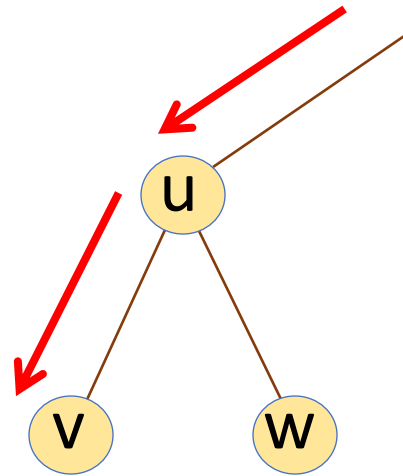
Depth-First Search (DFS)

tiempo: t



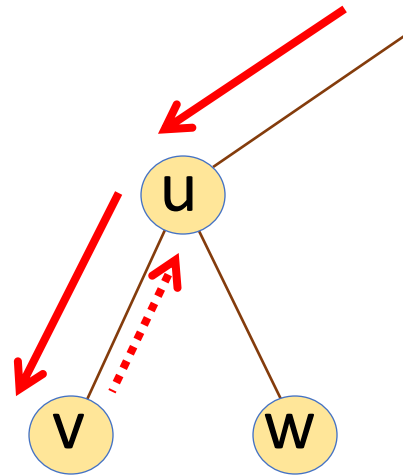
Depth-First Search (DFS)

tiempo: $t+1$



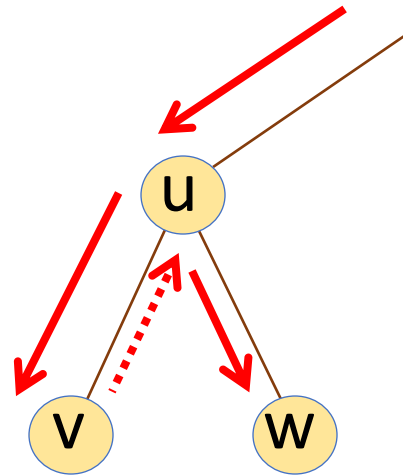
Depth-First Search (DFS)

tiempo: $t+2$



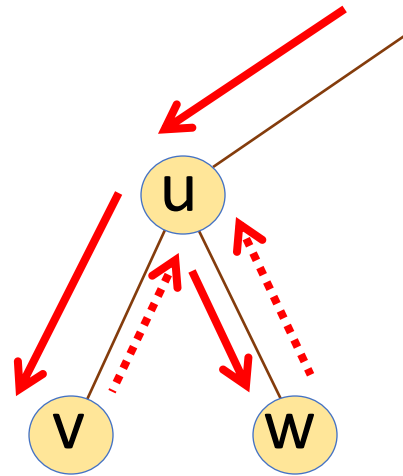
Depth-First Search (DFS)

tiempo: $t+3$



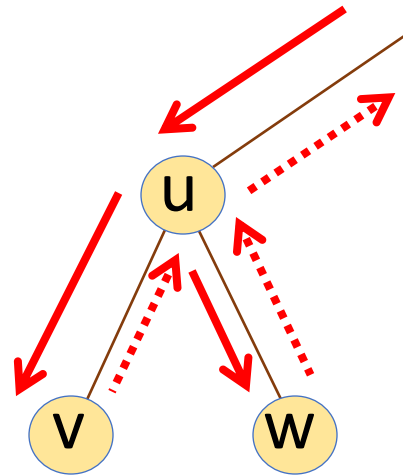
Depth-First Search (DFS)

tiempo: $t+4$

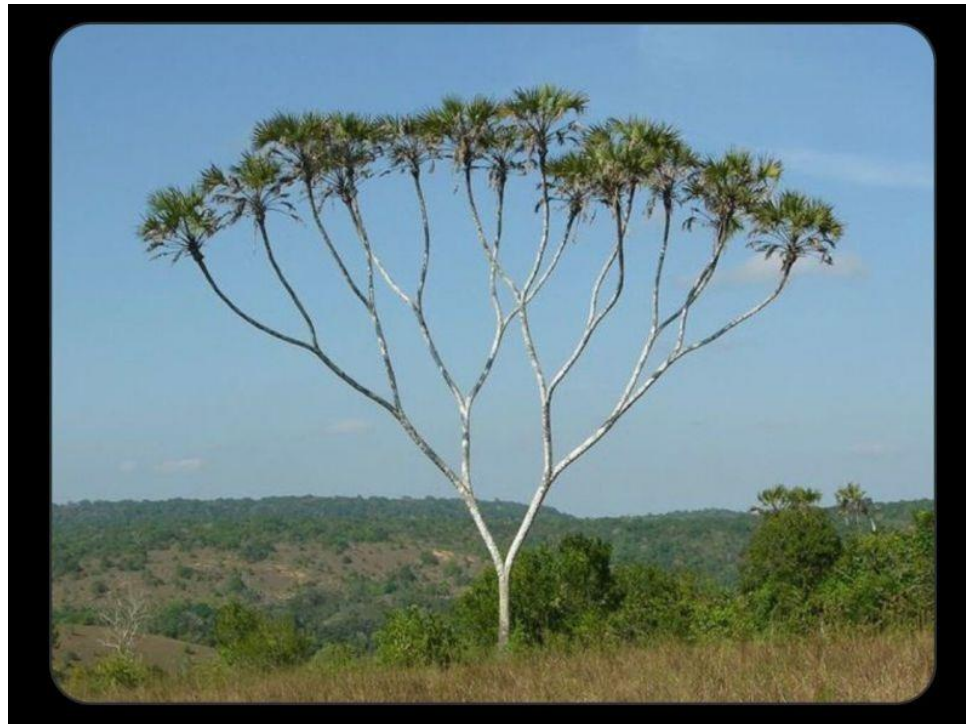


Depth-First Search (DFS)

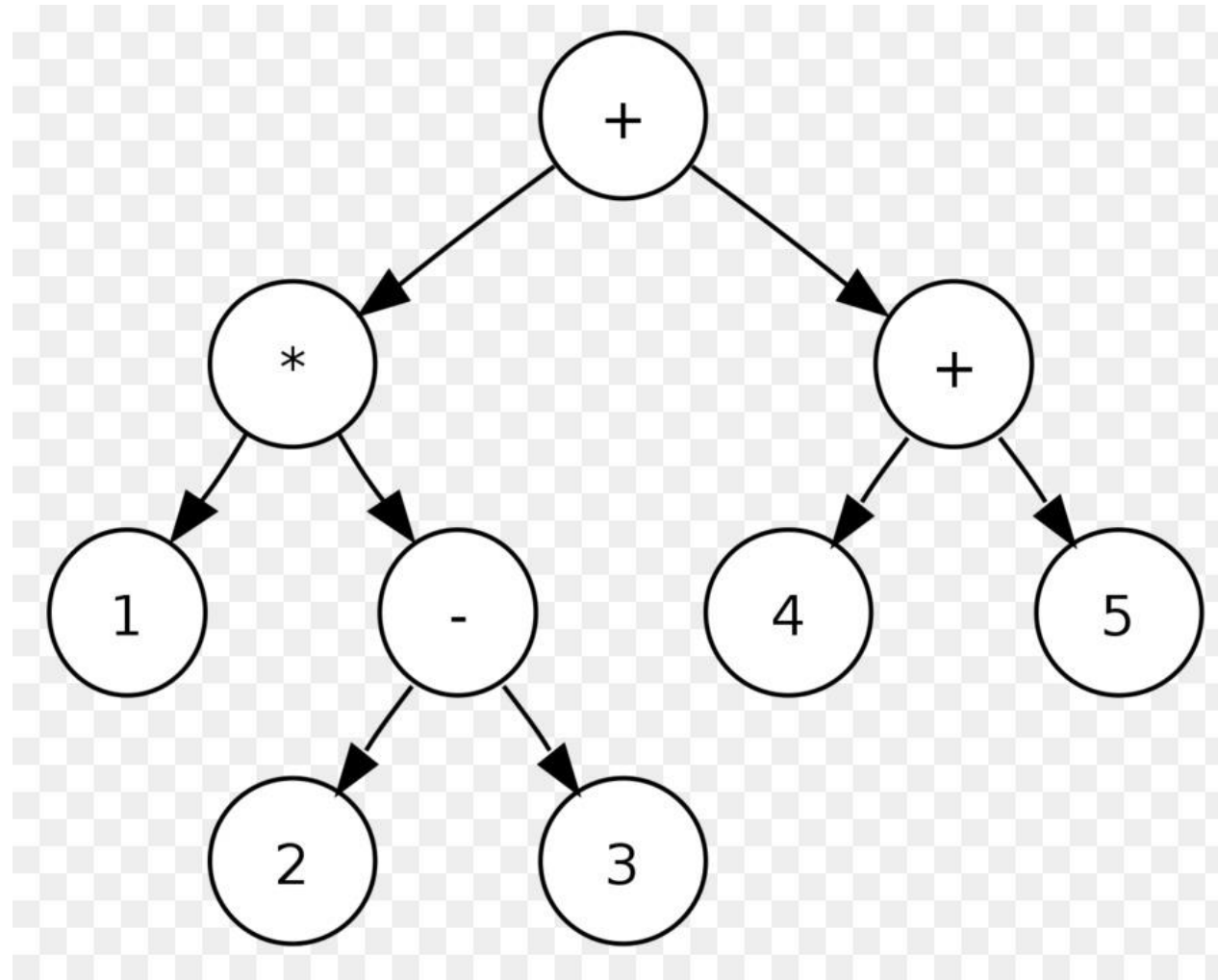
tiempo: $t+5$



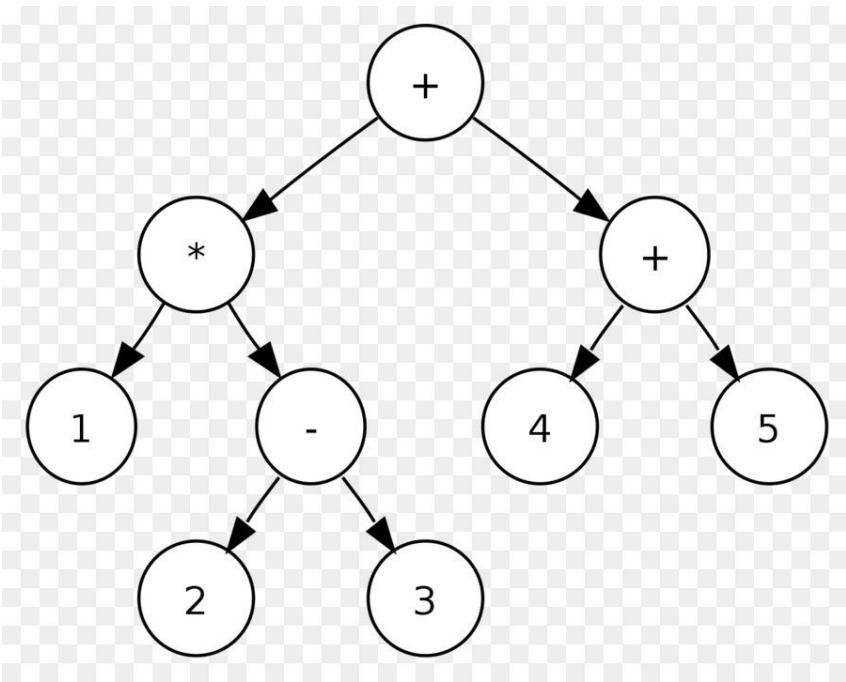
LoS árboles binarios, al parecer existen...



Así luce un árbol binario especial...



Recorriendo un árbol...



Una forma es llamada pre-orden:

- Recorrer subárbol izquierdo
- Recorrer raíz
- Recorrer subárbol derecho

$$1 * (2 - 3) + (4 + 5)$$

Búsqueda en grafos

- Los árboles son un caso particular de otras estructuras generales interesantes. Los grafos.
- Los grafos son estructuras que consisten de nodos y arcos (o aristas) que pueden conectar nodos.
- Se utilizan para representar diversos escenarios.

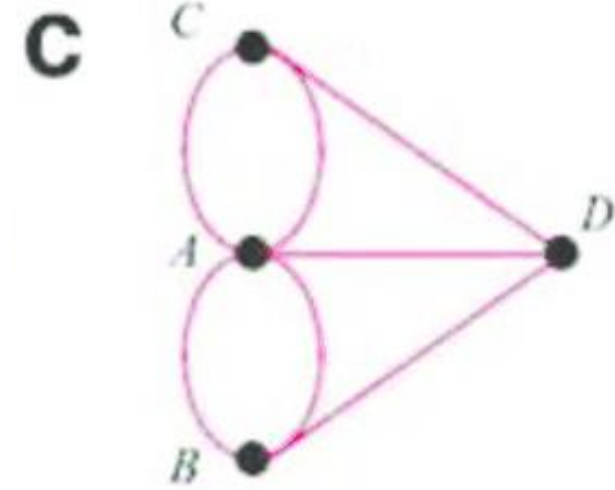
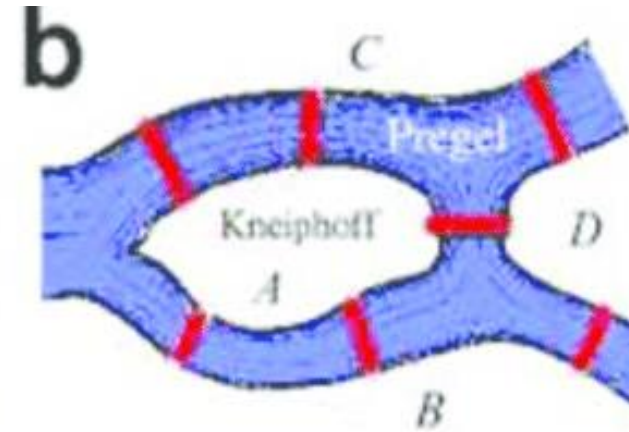
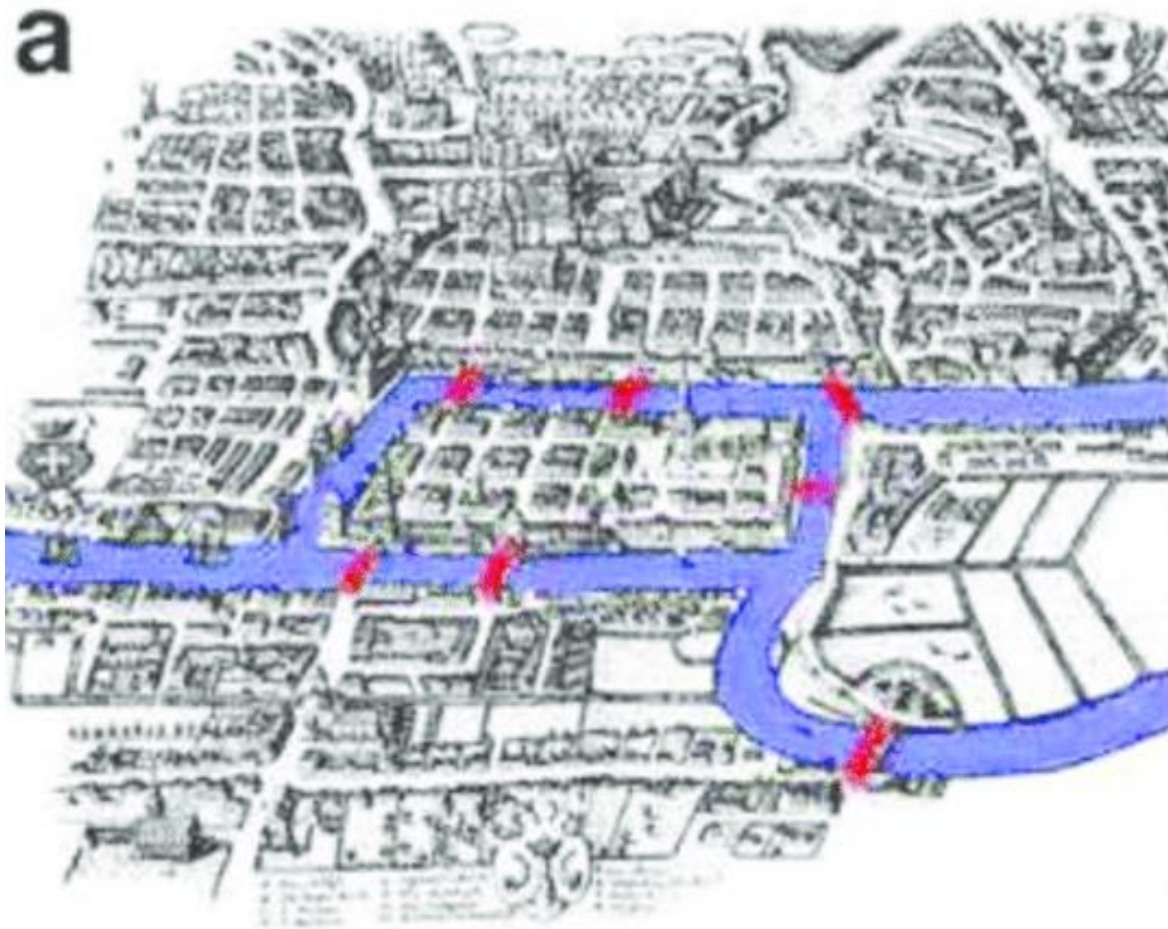
Esto es representable por medio de un grafo



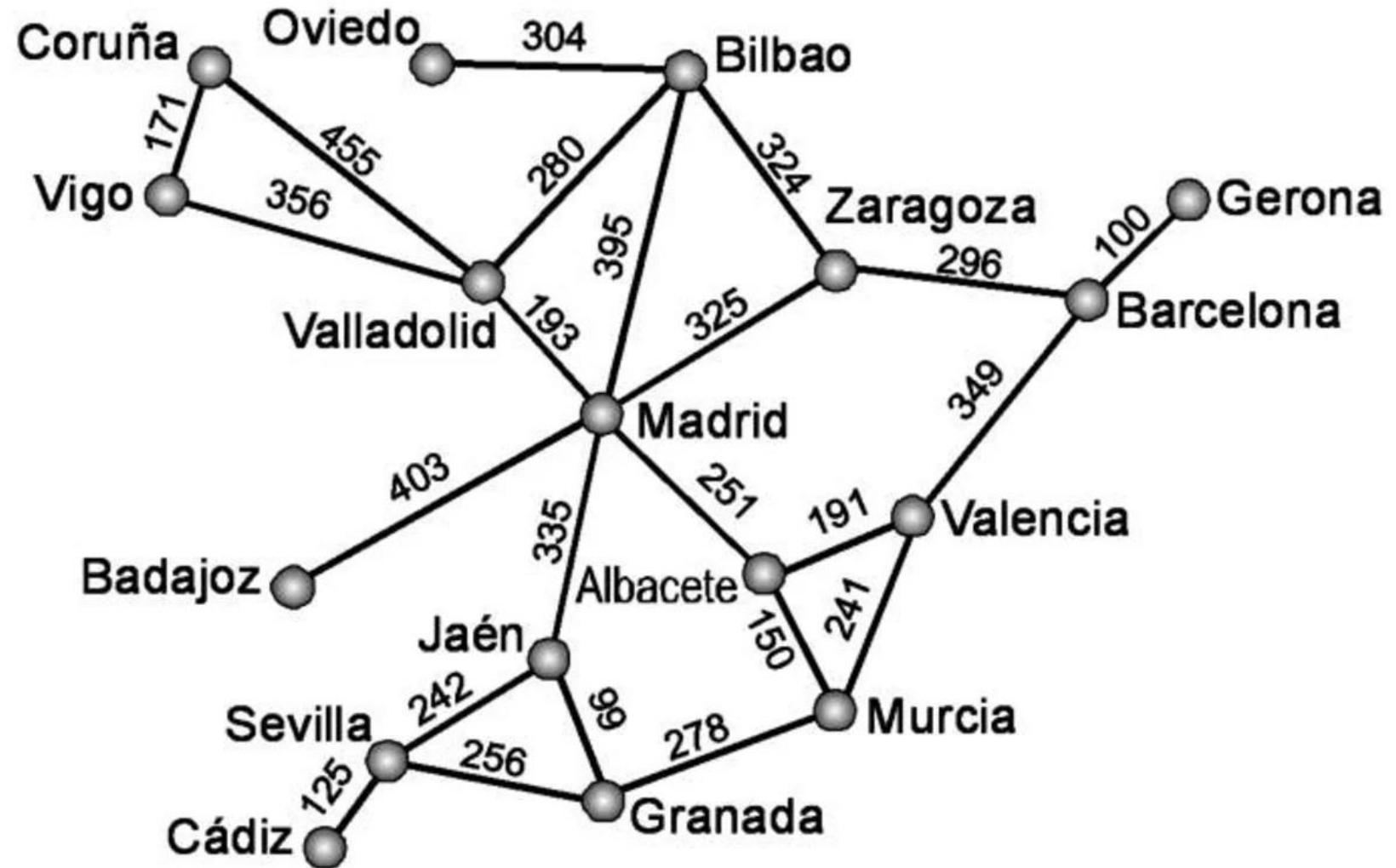
Todo
comienza
con Euler



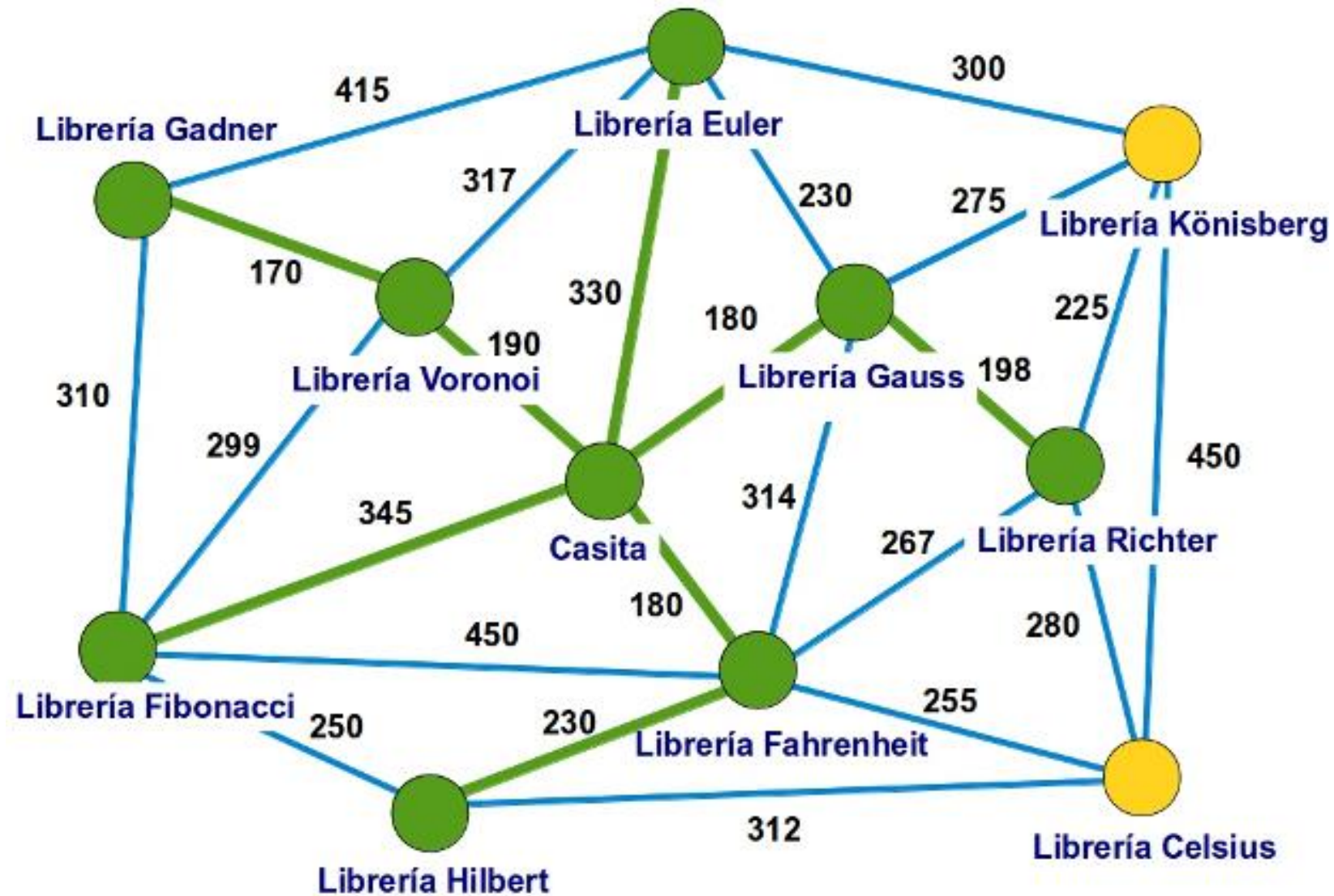
1736. Königsberg



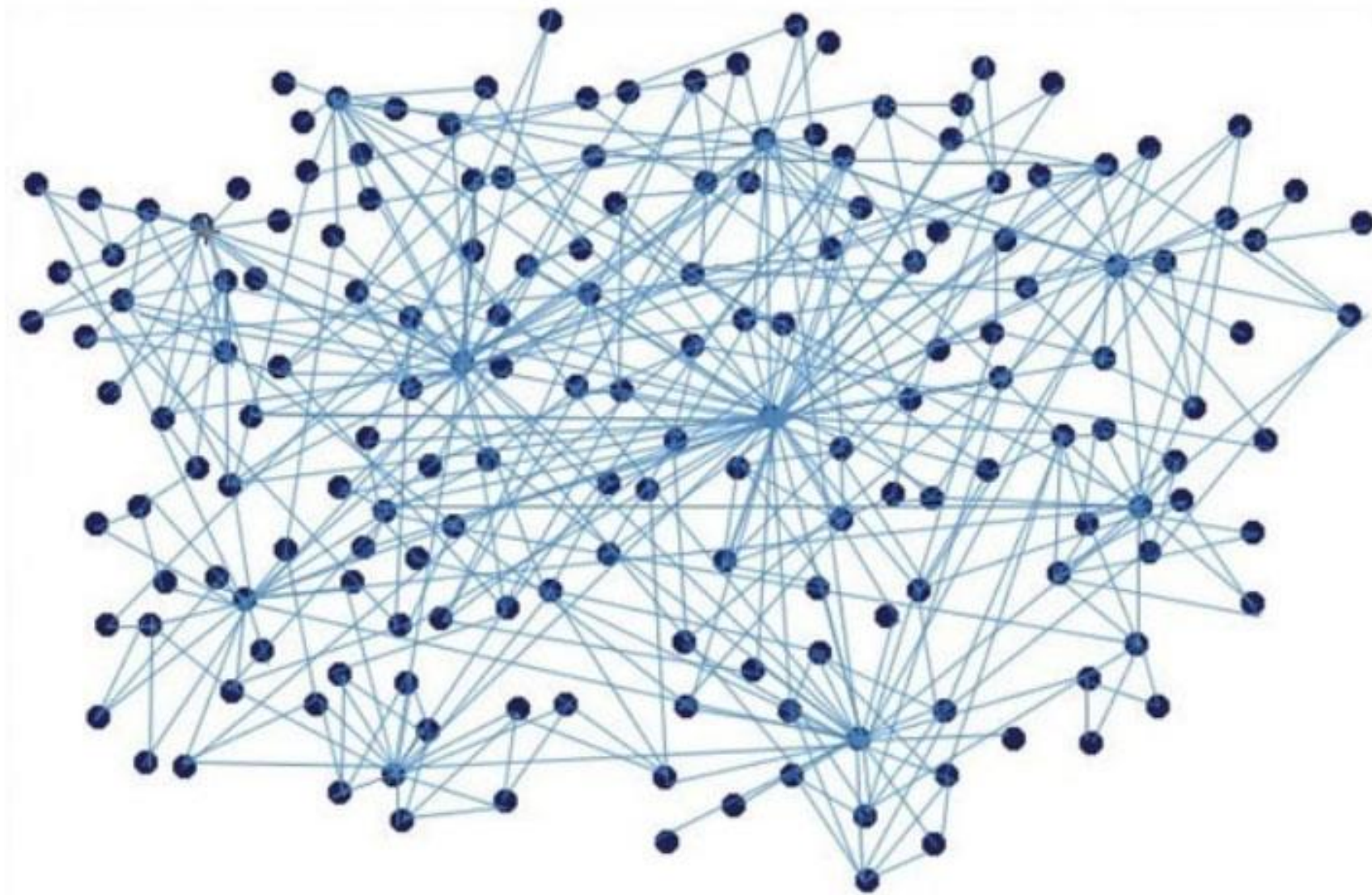
Ejemplo 1



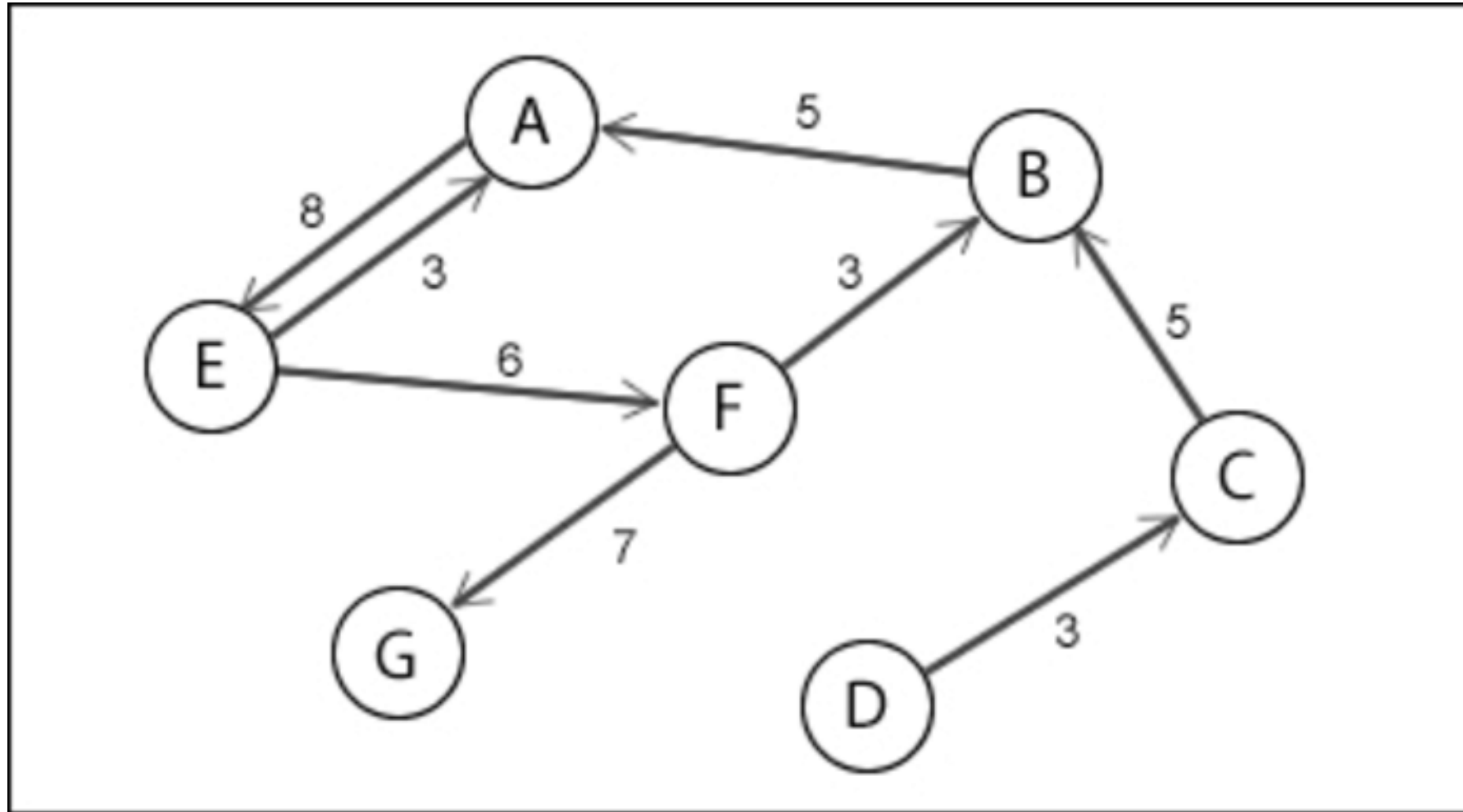
Ejemplo 2



Ejemplo 3 . Redes sociales



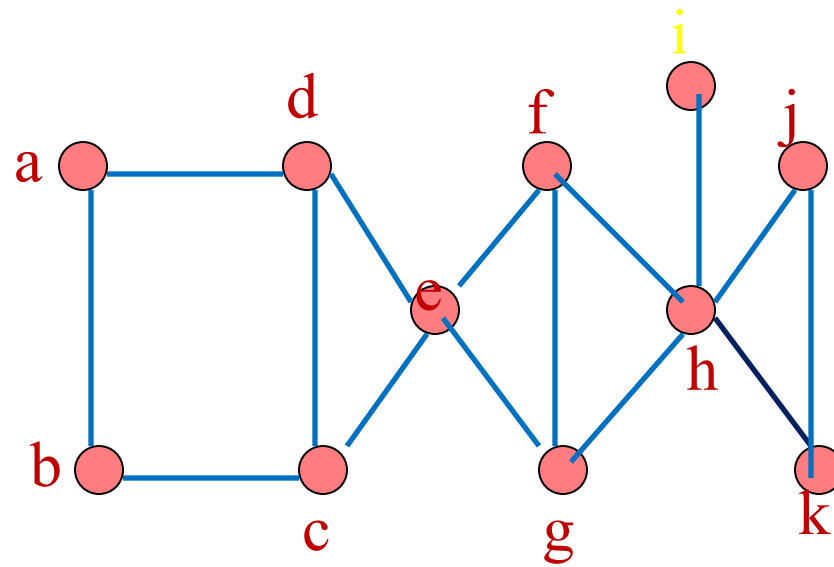
Ejemplo último. Redes dirigidas

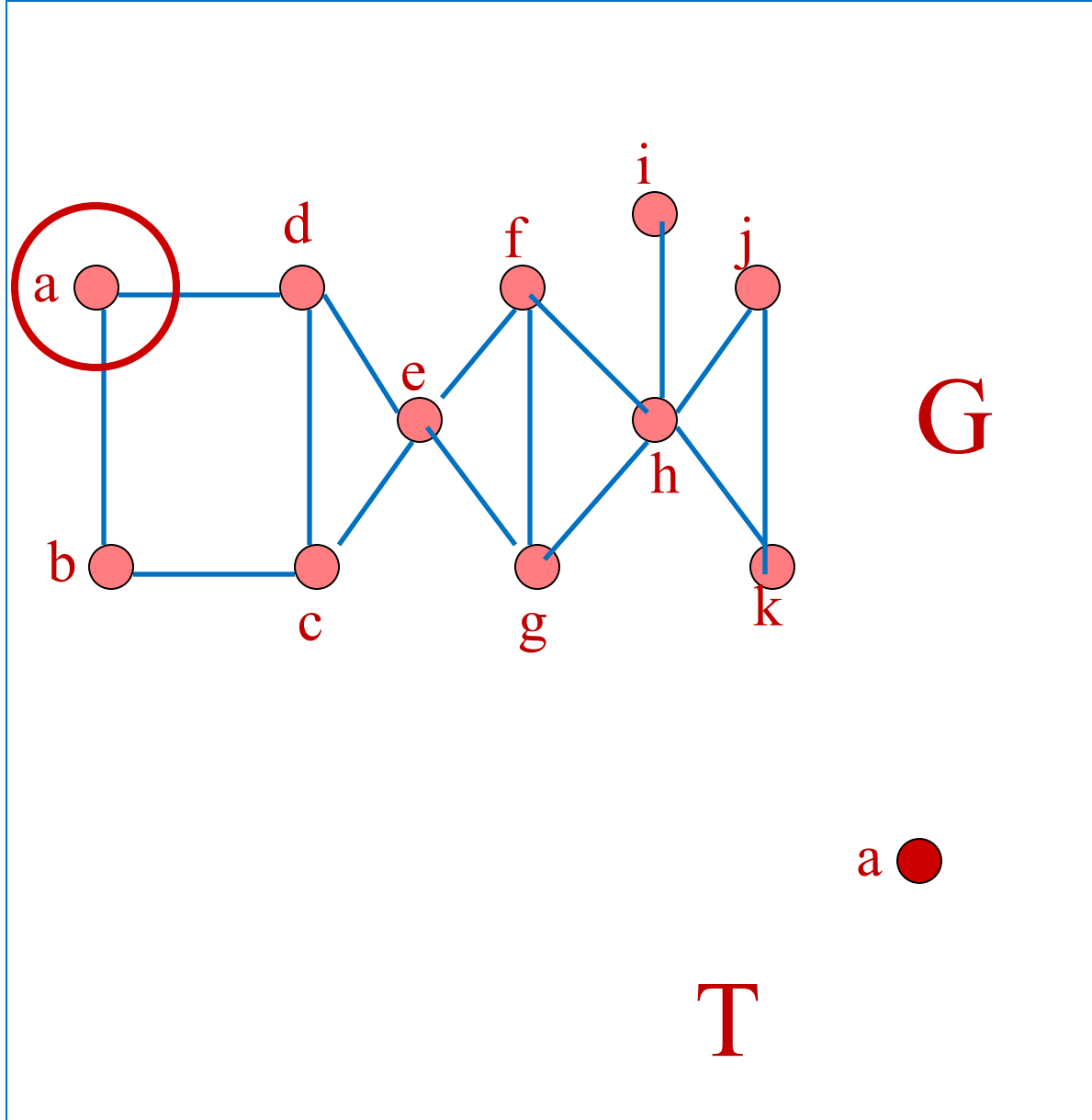


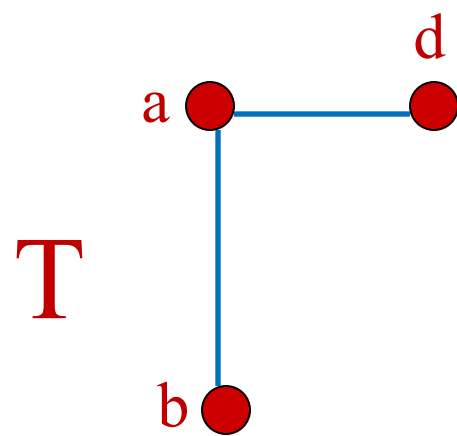
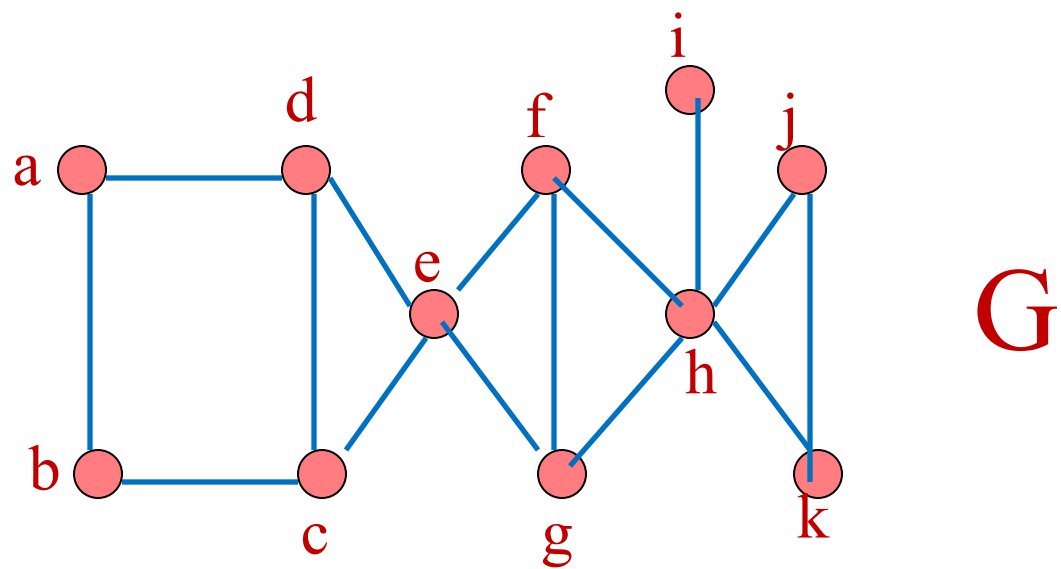
Breadth First Search

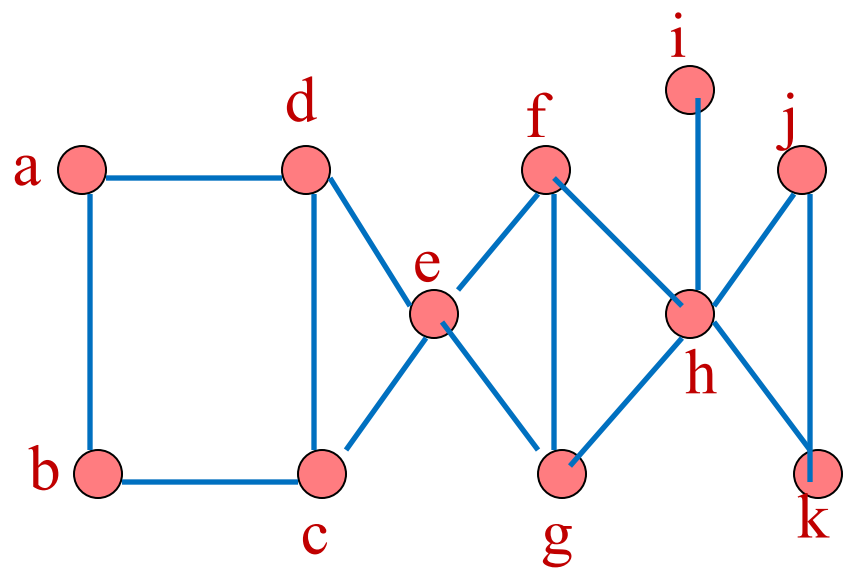
- Entrada: Grafo conexo con vértices $v_1, v_2, \dots, v_n$.
- Salida: T , árbol de cobertura de G .
- Algoritmo:
 - 1. (Inicio) Sea v_1 la raíz de T , sea $V = \{v_1\}$
 - 2. (Agrega aristas) Para cada vértice $x \in V$, agregar la arista $\{x, v_k\}$ a T , donde k es el menor índice tal que al agregar la arista $\{x, v_k\}$ a T no se forma un ciclo. Si no hay más aristas para agregar, parar. T es el árbol de cobertura.
 - 3. (Actualizar) Reemplazar V por los hijos v en T de los vértices x de V donde las aristas $\{x, v\}$ se agregaron en el paso 2. Repetir paso 2 para el nuevo conjunto V .

Ejemplo:



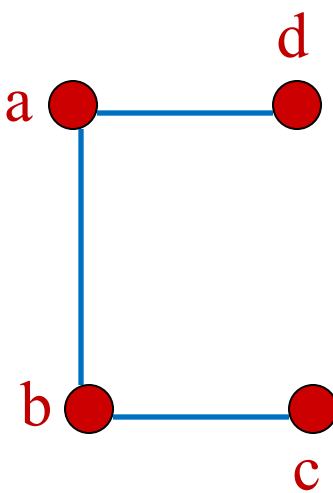


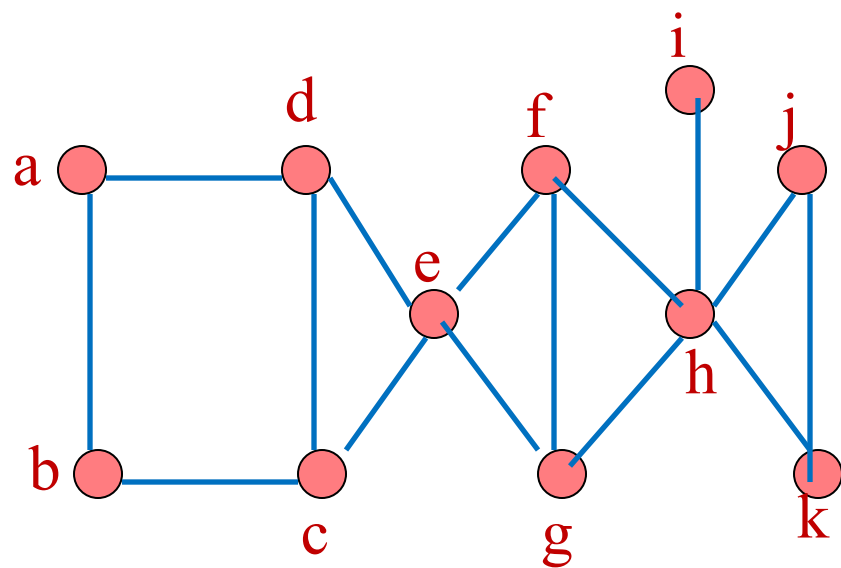




G

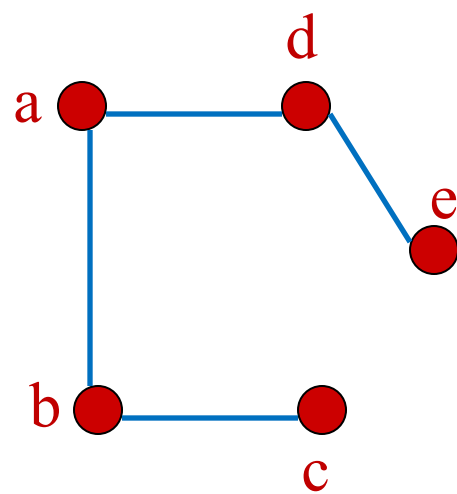
T

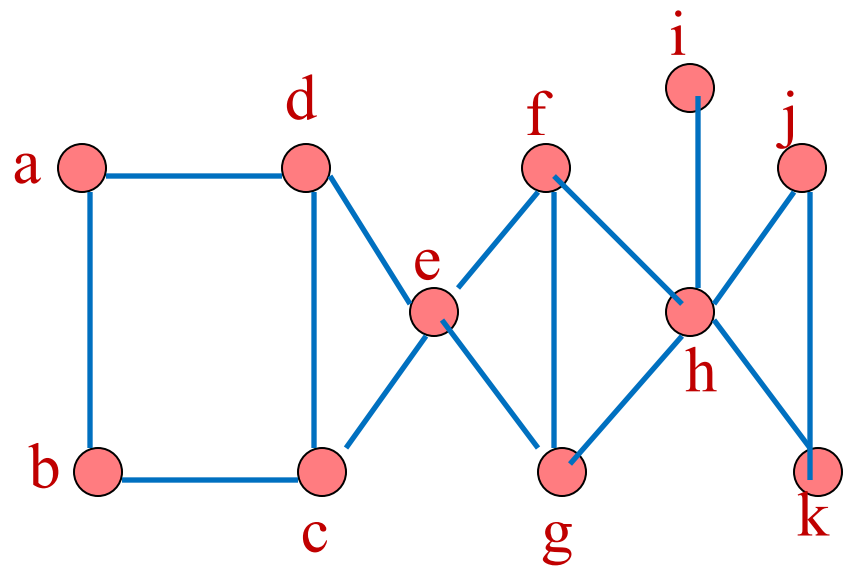




G

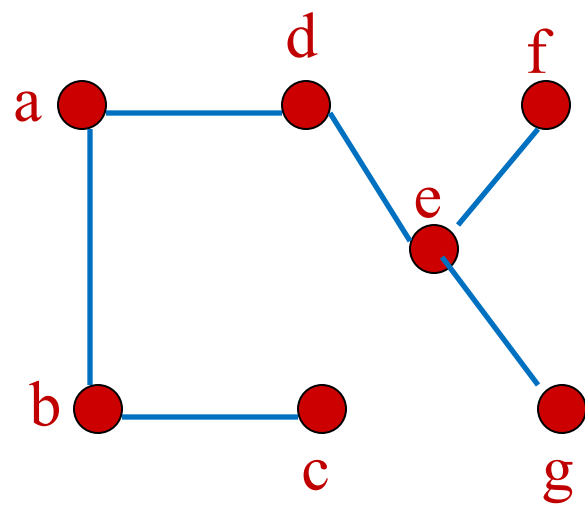
T

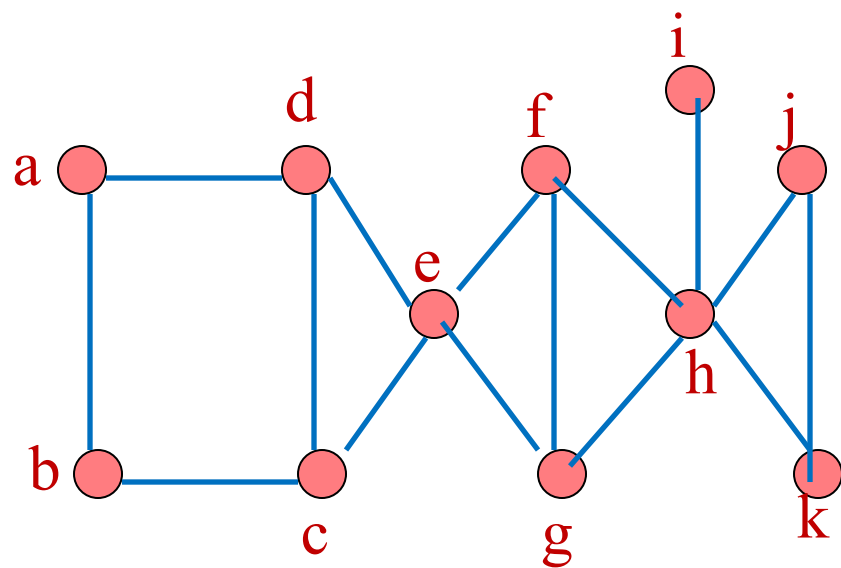




G

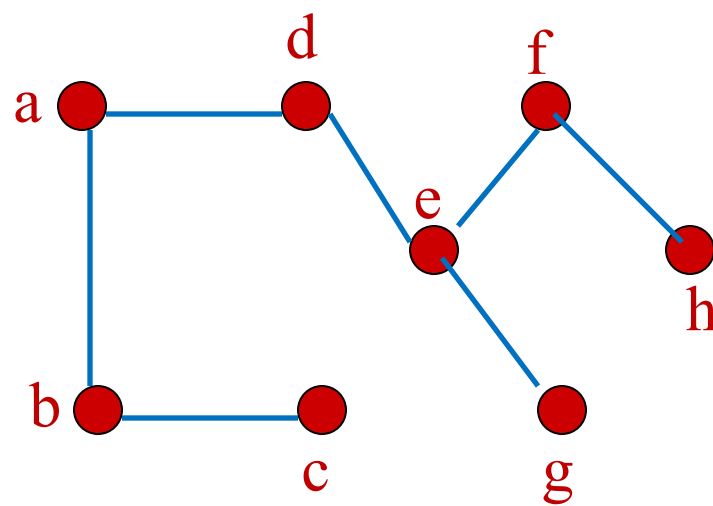
T

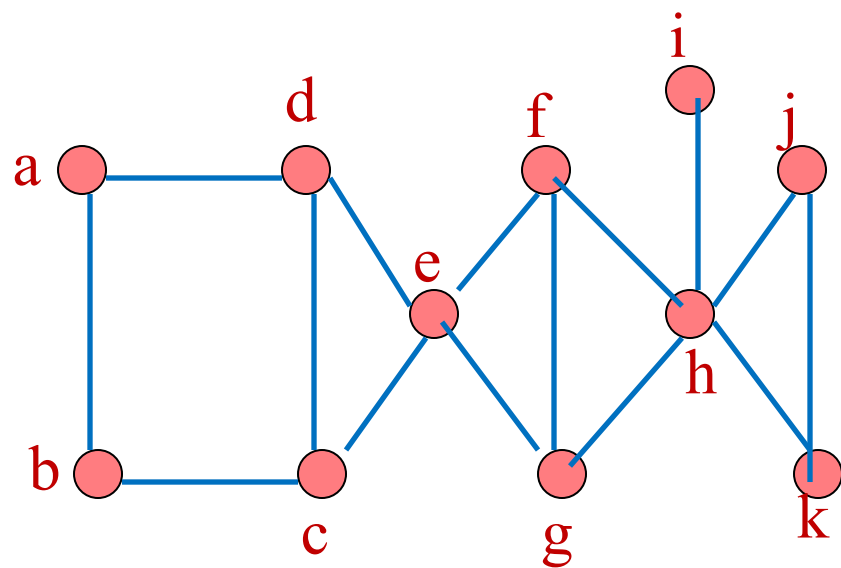




G

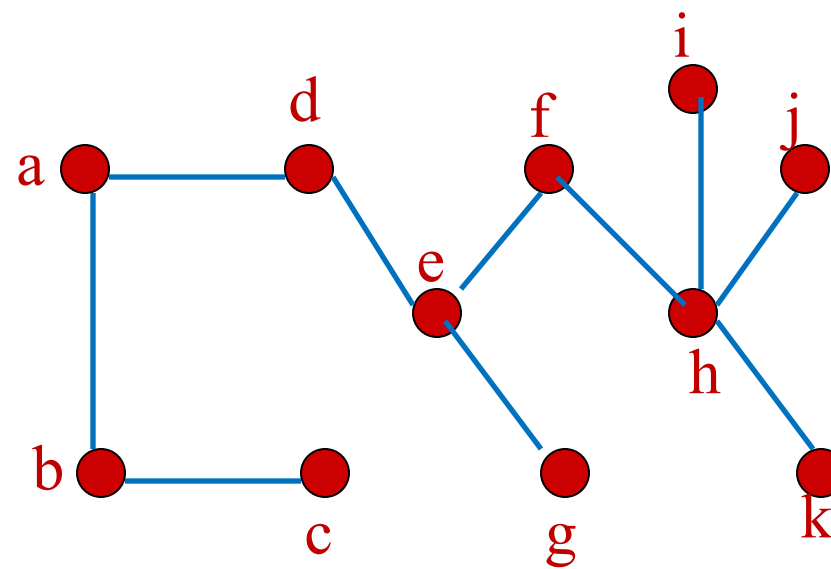
T





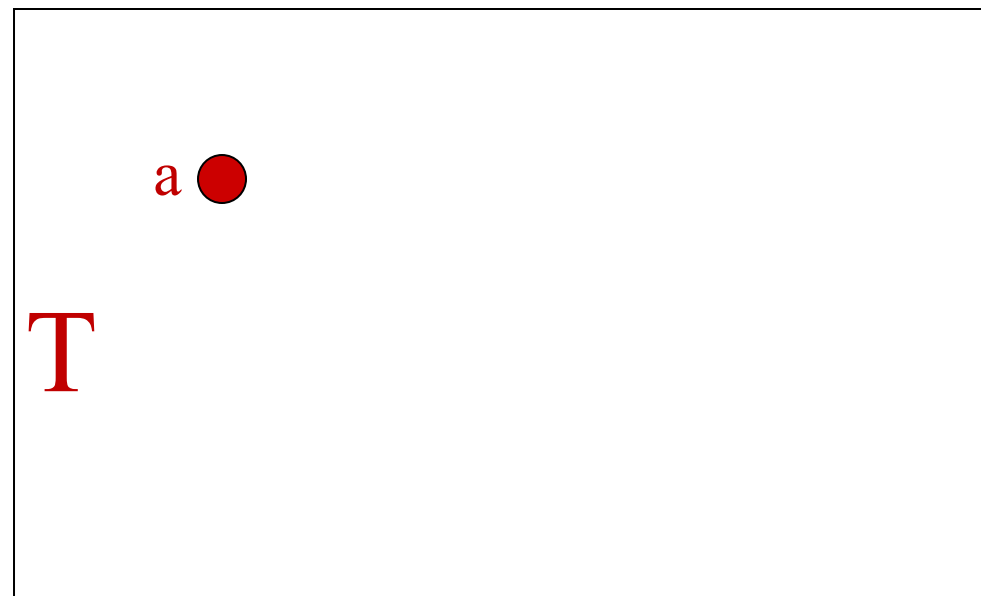
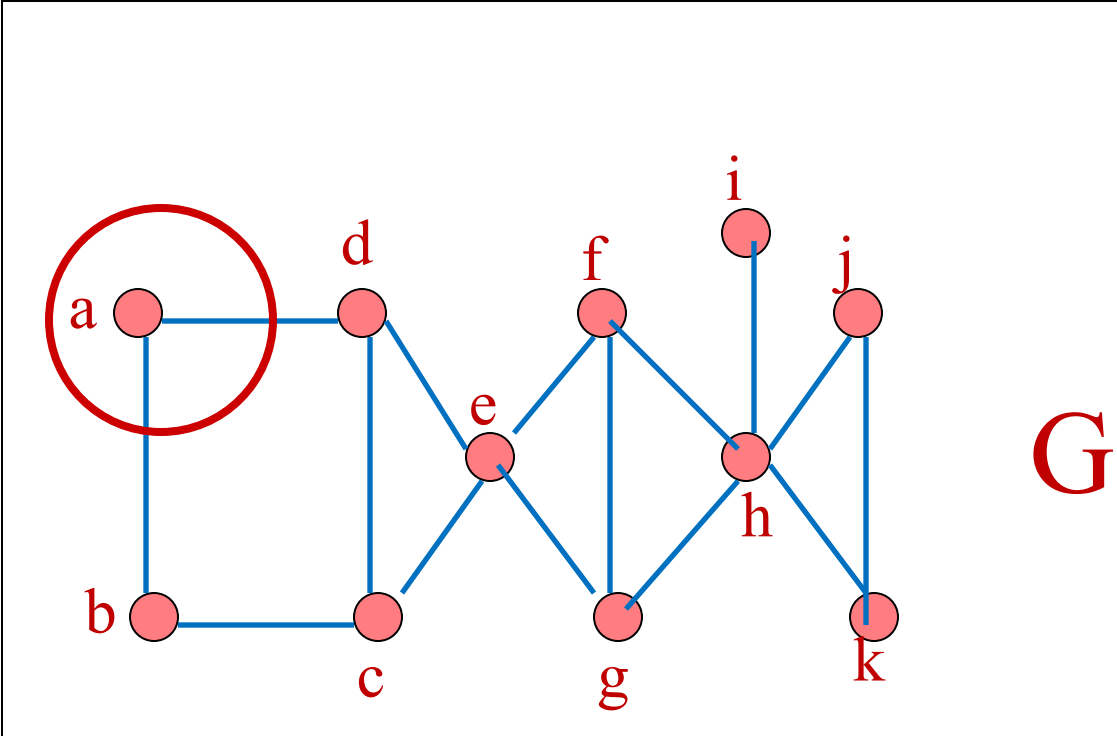
G

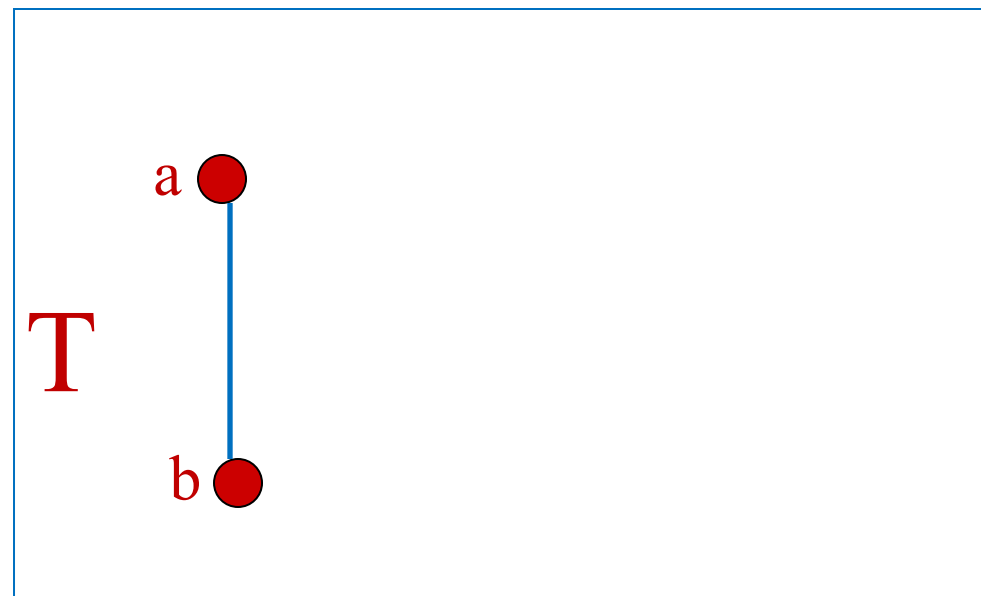
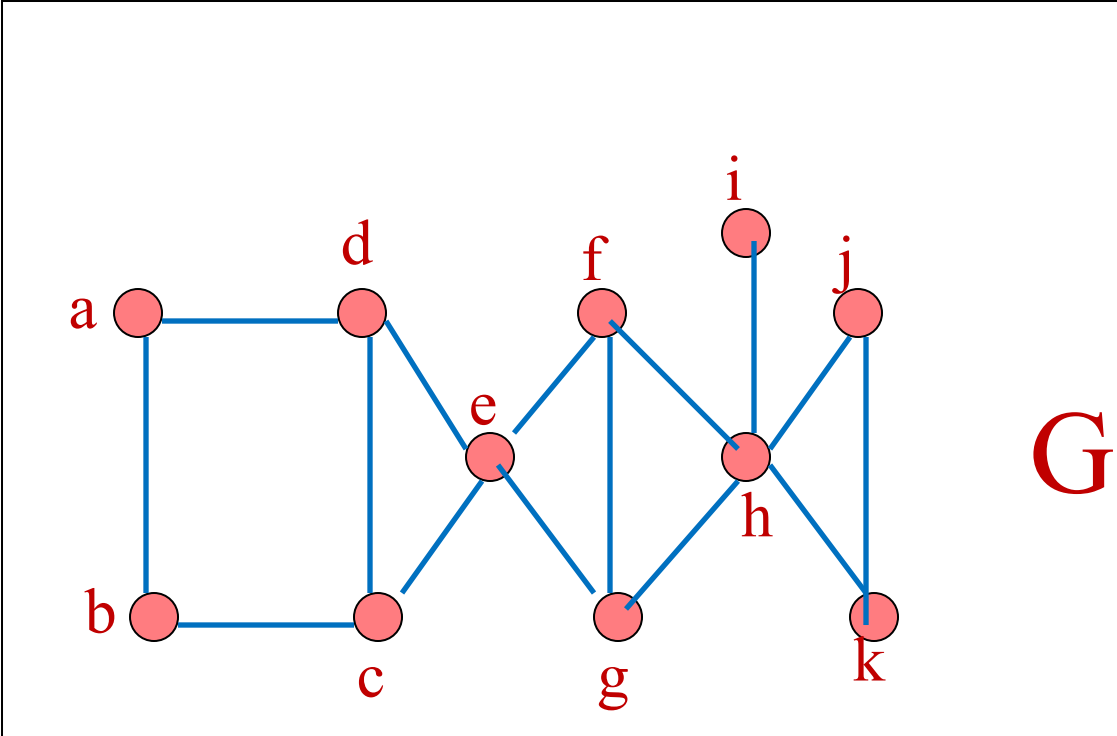
T

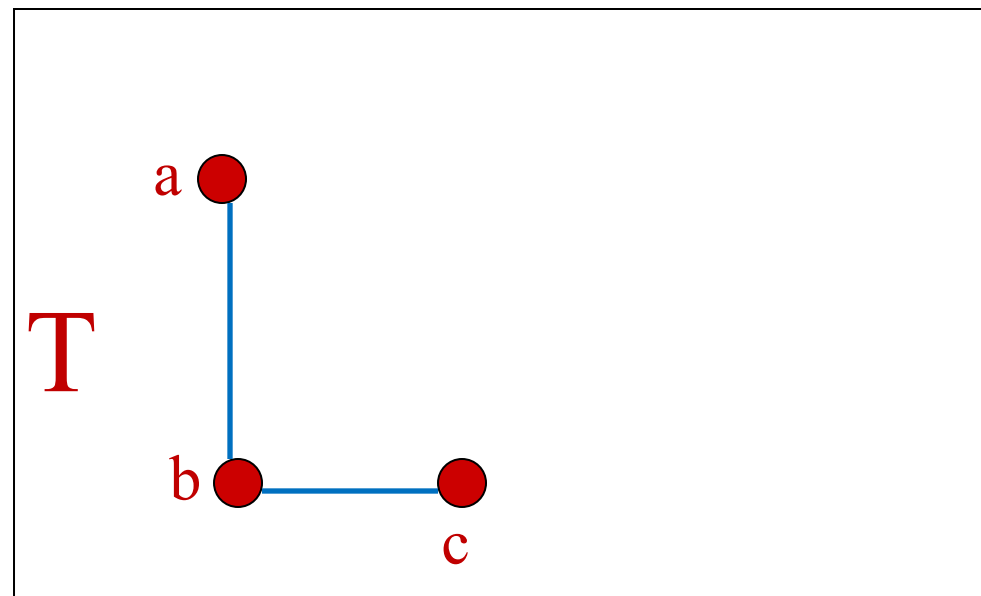
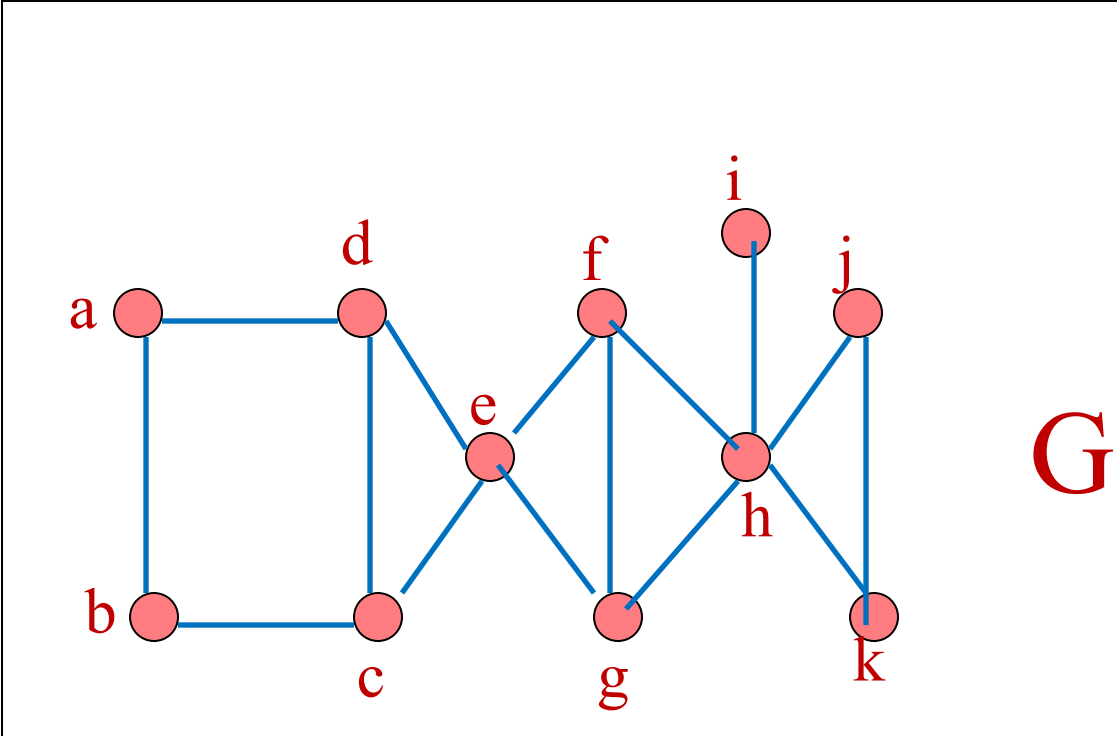


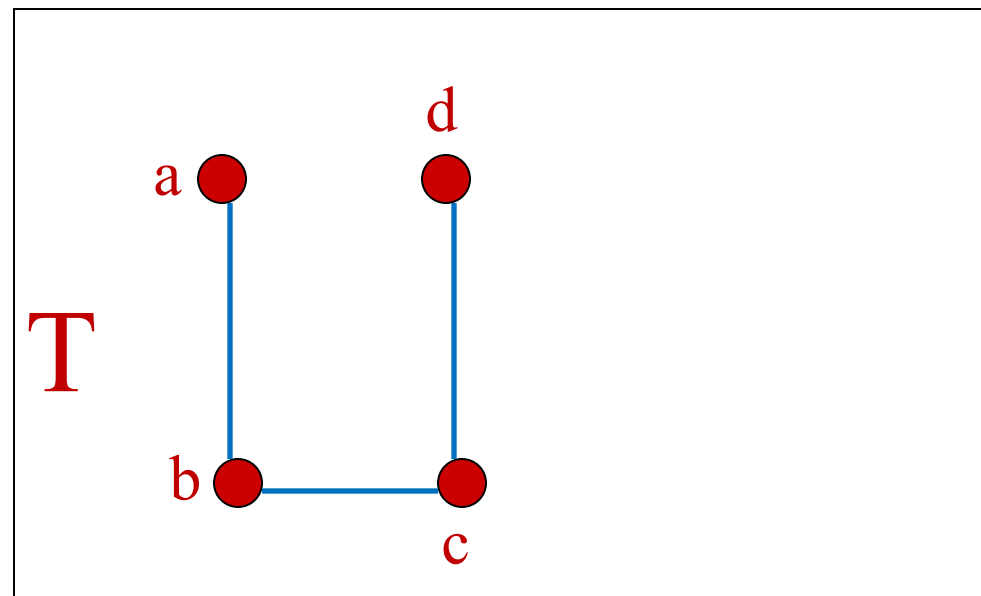
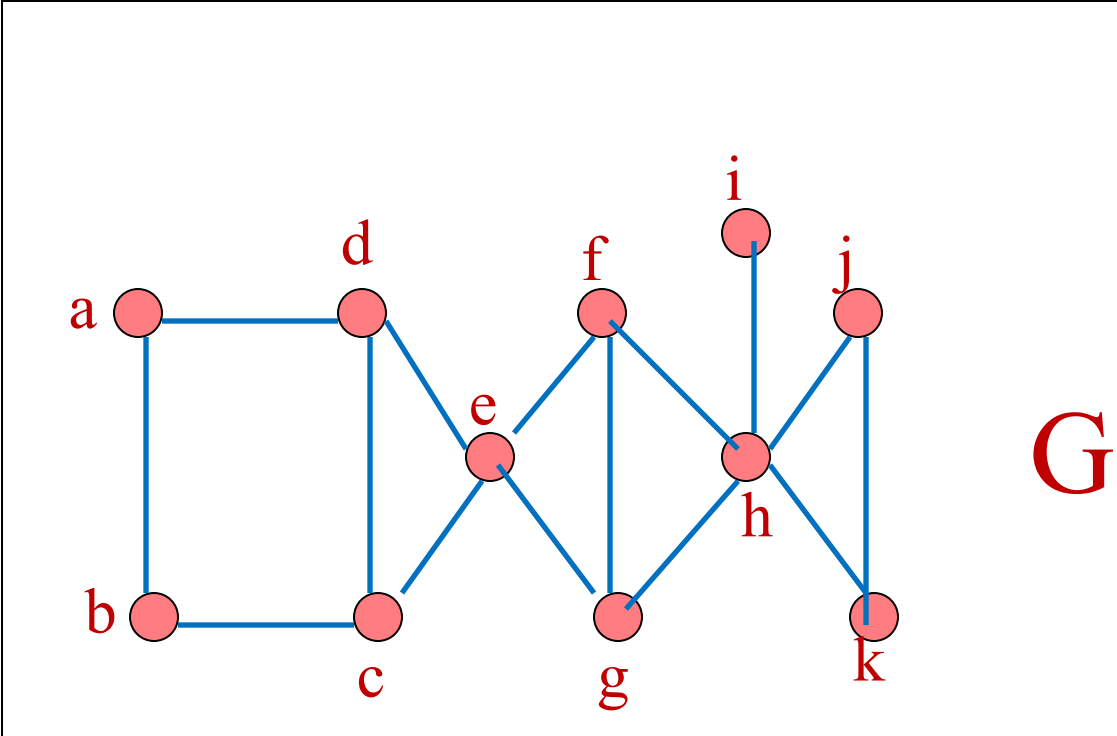
Depth First Search

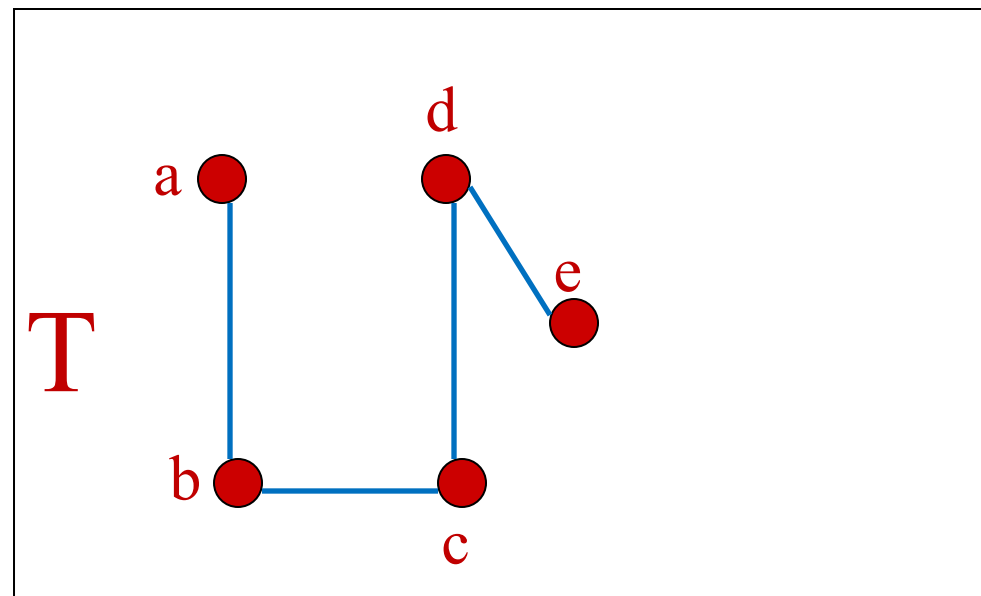
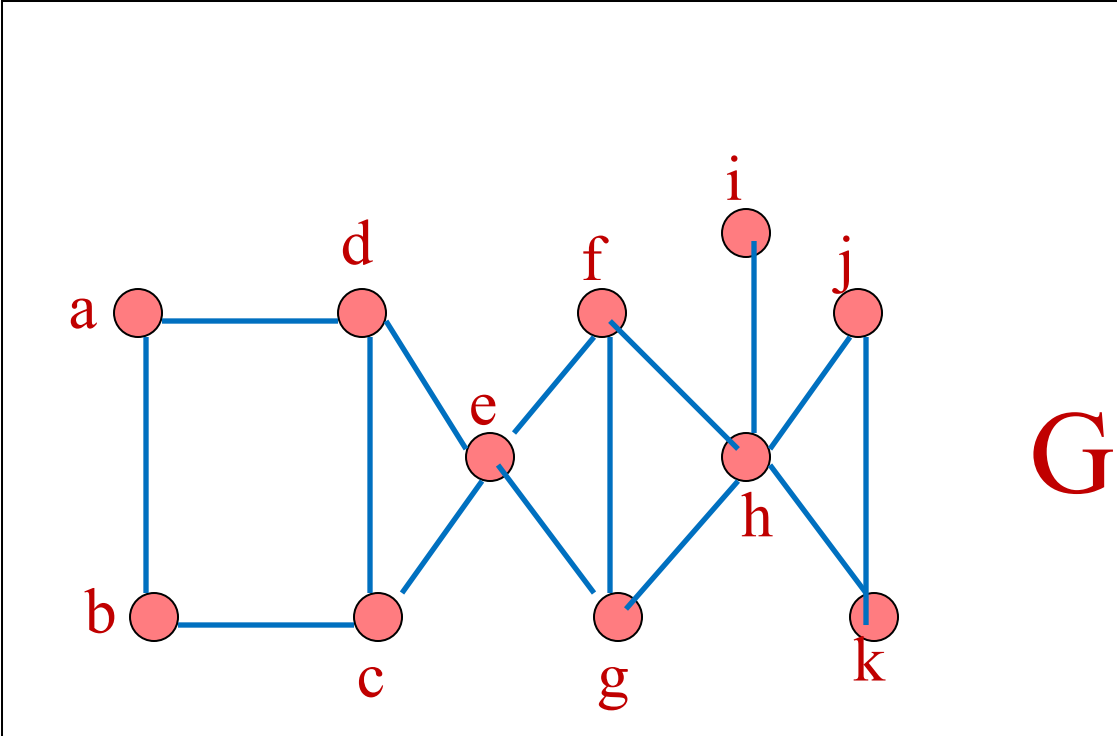
- Entrada: Grafo conexo con vértices $v_1, v_2, \dots, v_n$.
- Salida: T , árbol de cobertura de G .
- Algoritmo:
 - 1. (Visita vértice) Sea v_1 la raíz de T , sea $L = \{v_1\}$
 - 2. (Encuentra aristas y vértices no visitados adyacentes a L) Para todos los vértices adyacentes a L , elegir arista $\{L, v_k\}$ donde k es el menor índice tal que al agregar la arista $\{L, v_k\}$ a T no se forma un ciclo. Si no hay tal arista, ir a paso 3; en caso contrario agregar $\{L, v_k\}$ a T y hacer $L = v_k$. Repetir paso 2 con el nuevo valor de L
 - 3. (Backtrack o fin) Si x es padre de L en T , hacer $L = x$ y aplicar paso 2 al nuevo valor de L . Si L no tiene padres en T (o sea, $L = v_1$), terminar.

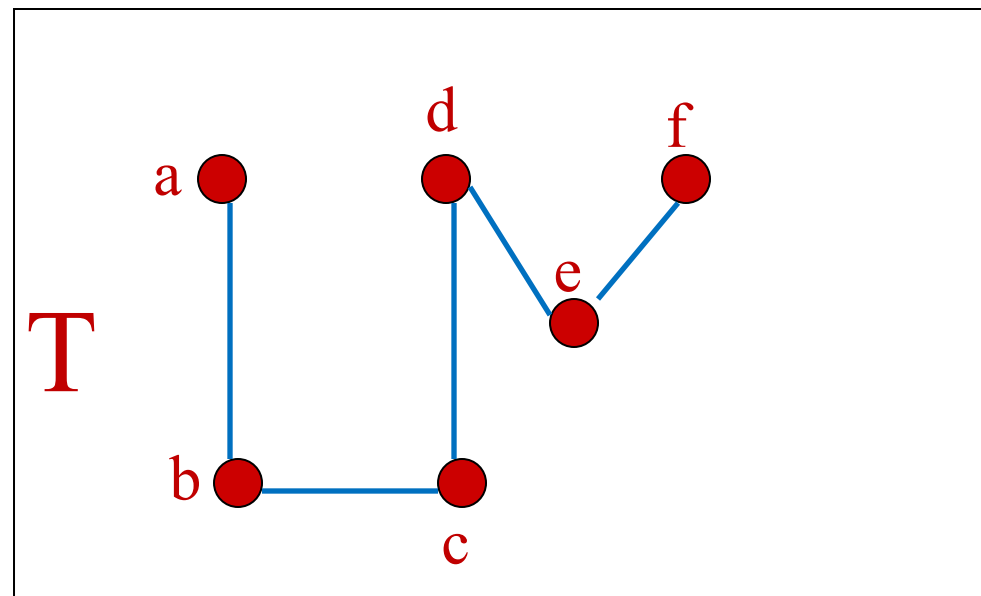
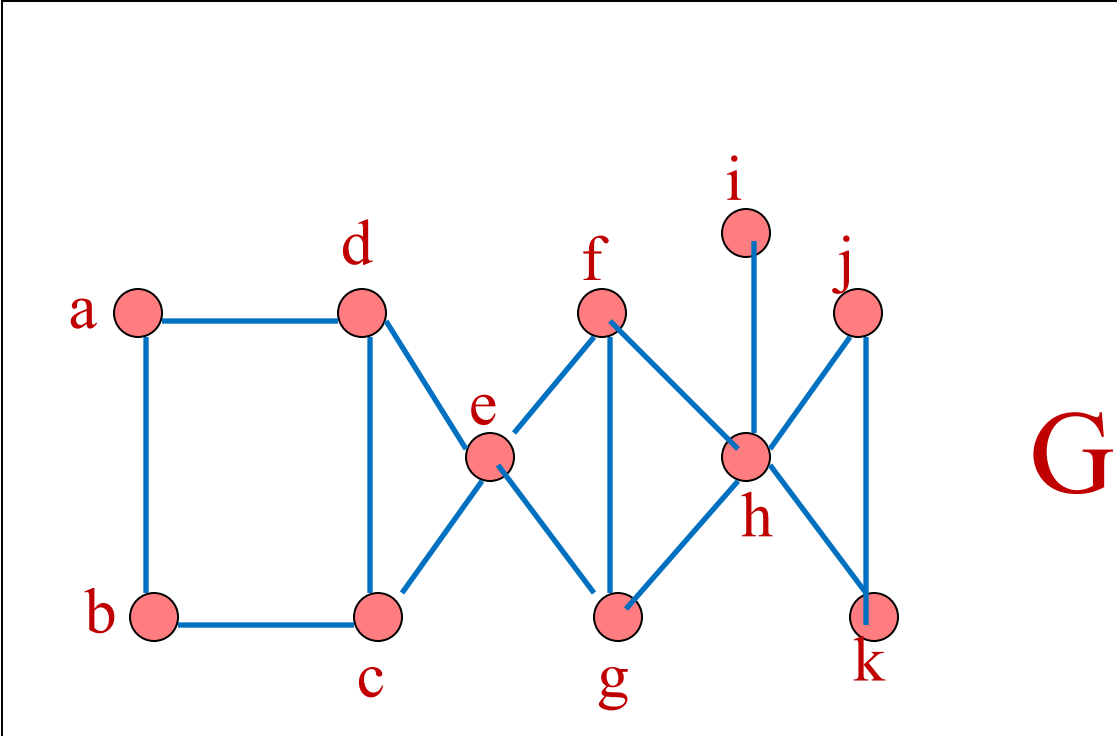


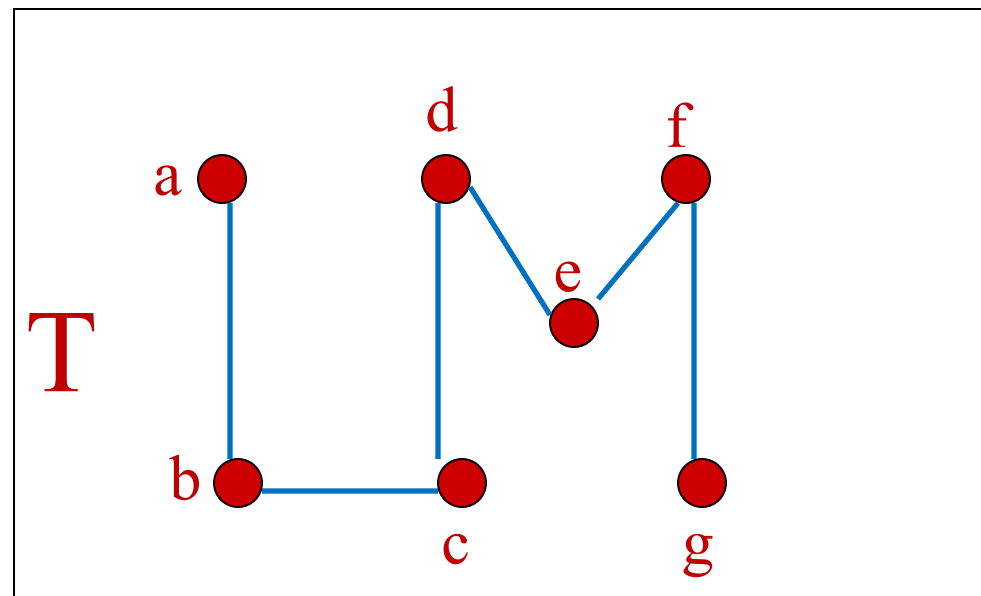
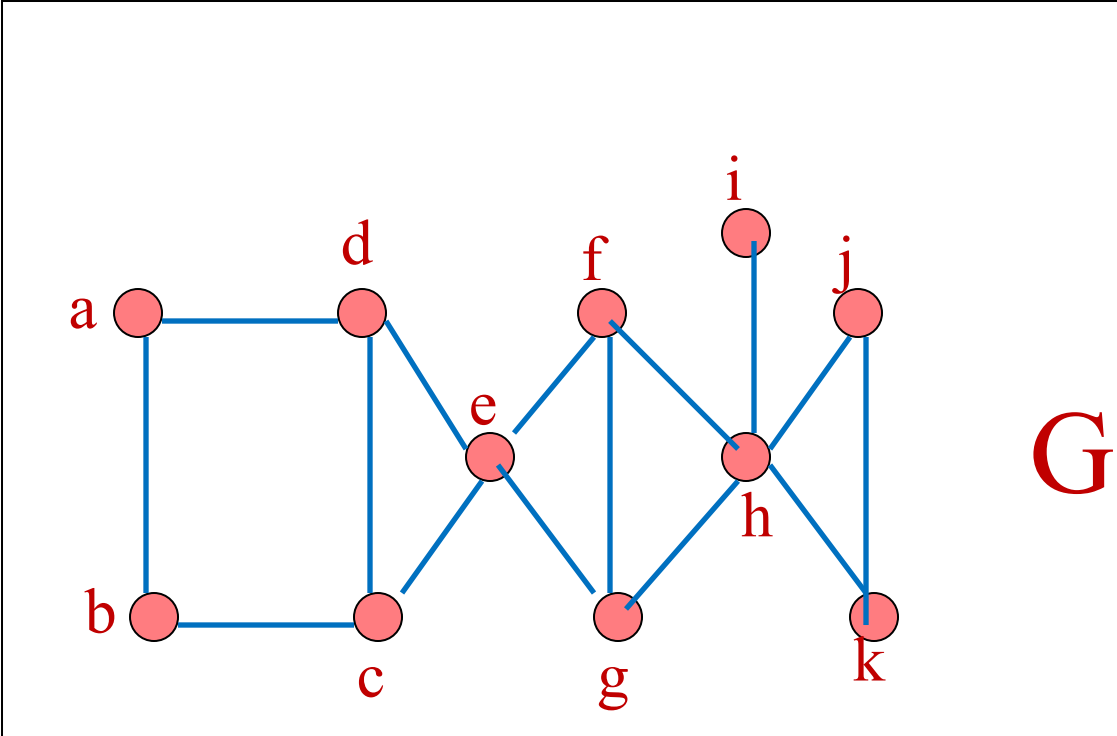


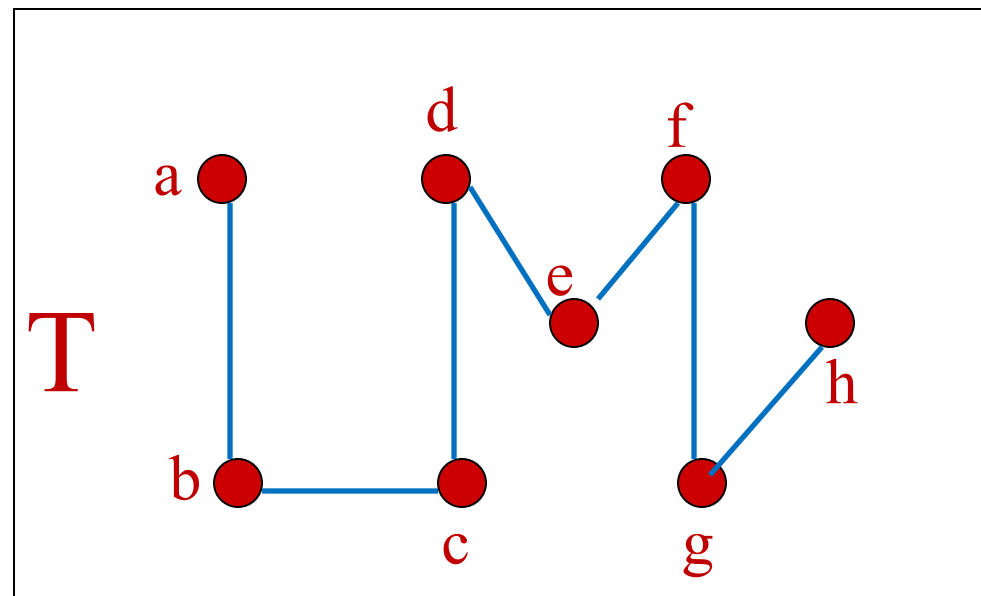
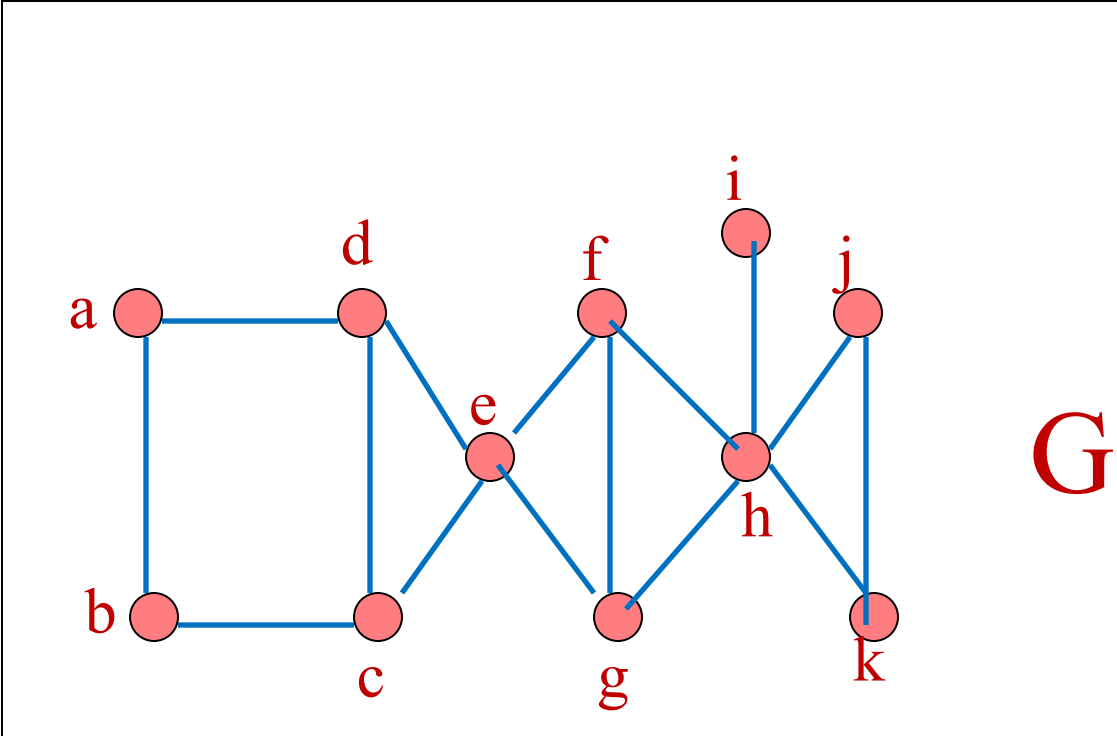


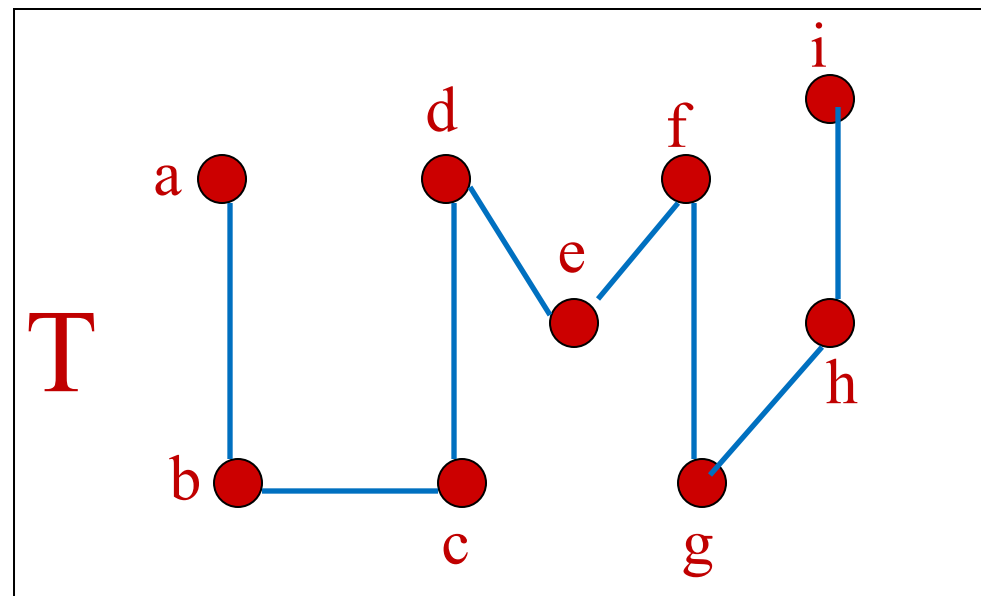
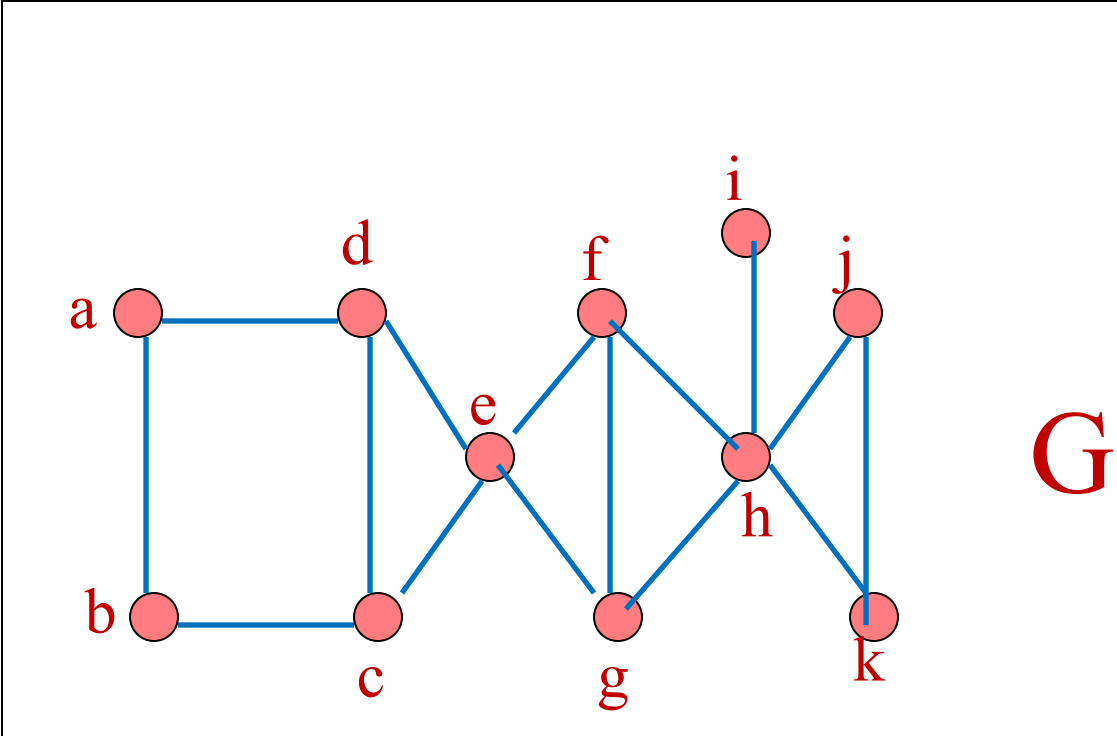


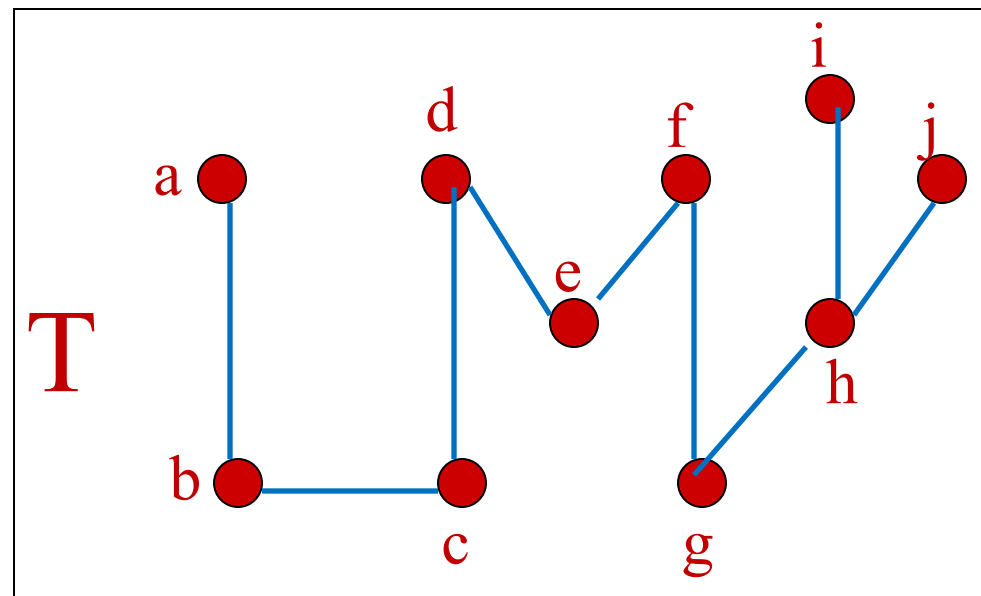
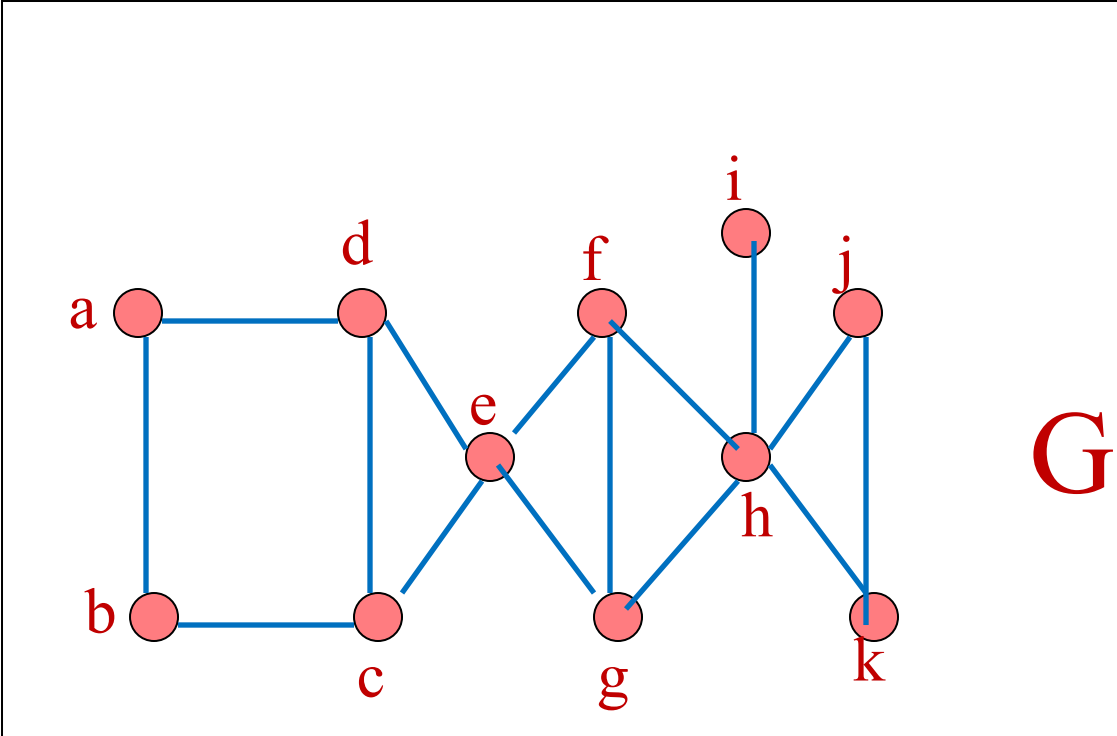


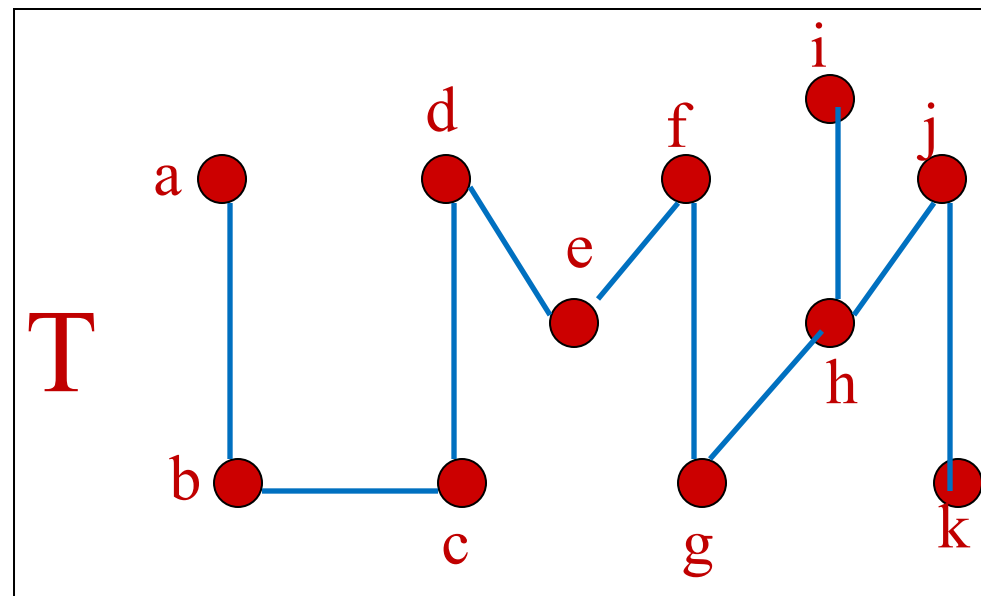
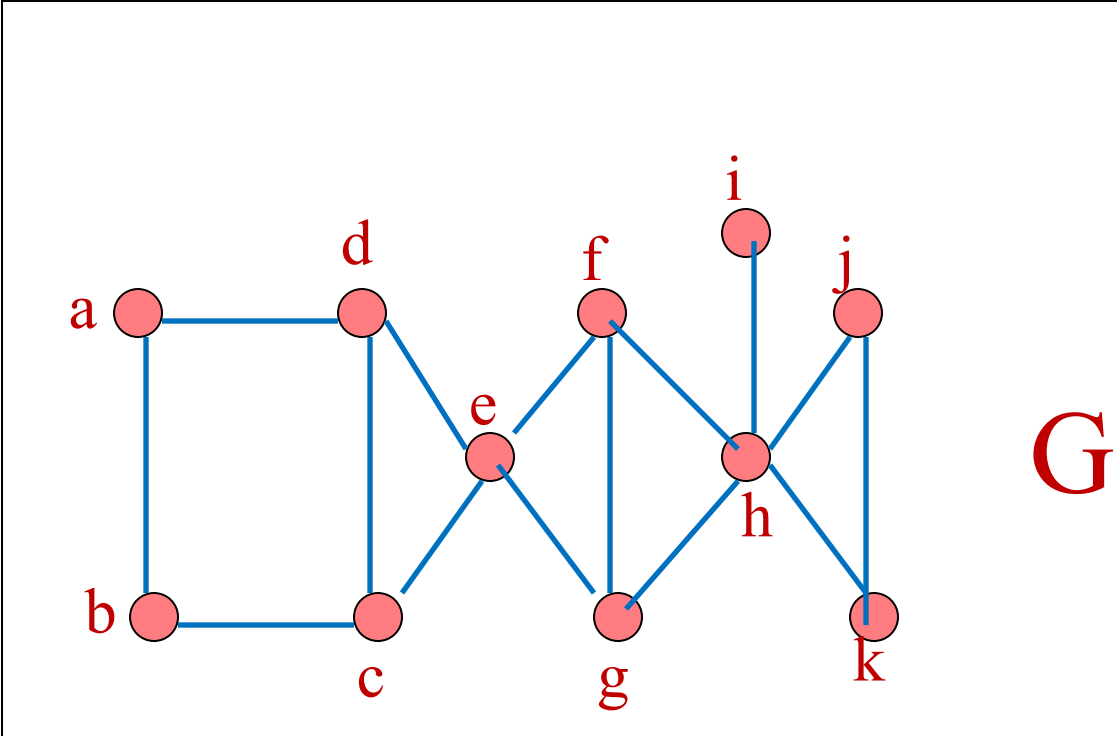












Un pequeño cambio de ritmo

La tarea

¿Conversemos de la tarea?

A partir de un árbol genealógico realista, ustedes deberán implementar un algoritmo de búsqueda en anchura y uno de búsqueda en profundidad, para recorrer ese árbol y tener así un listado de toda la parentela.

Para no complicarnos la vida, este árbol tendrá como raíz la abuela materna (en los grupos de la tarea se pondrán de acuerdo para elegir la abuela desde la que considerarán los descendientes).

Elijan una abuela que no sea muy prolífica, con unos 20 a 25 parientes ya será suficiente (pueden podar el árbol si empieza a crecer mucho o si no quieren incorporar a algún pariente...)

¿Conversemos de la tarea? (Cont.)

Se trata de un trabajo grupal (aproximadamente 4 personas por grupo)

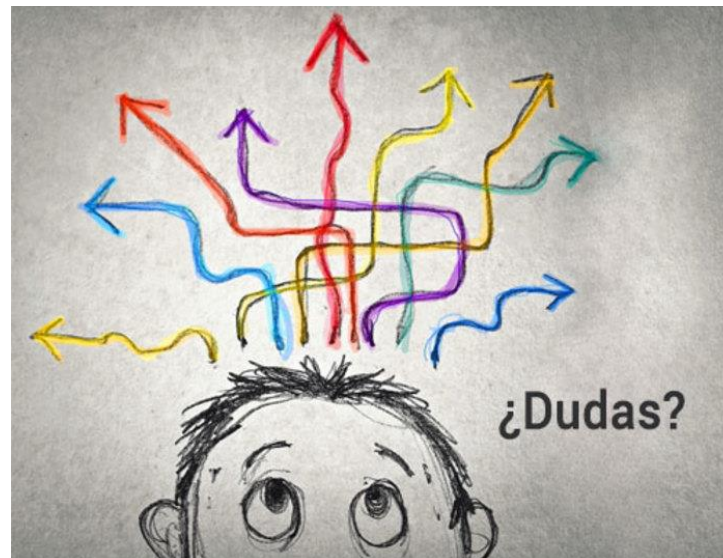
Alternativa 1. Búsqueda en anchura (obtendrán una lista de familiares)

Alternativa 2. Búsqueda en profundidad (obtendrán una lista distinta de familiares)

Entiendo que Python es el lenguaje que ya conocen. En la red encuentran los códigos ya operativos, pueden usarlos.

Fechas importantes

- Entrega de la tarea (en formato pdf, el código puede estar asociado a un link): 7 de julio
- Control: 7 de julio (segunda mitad de la clase)
- ¿Consultas?

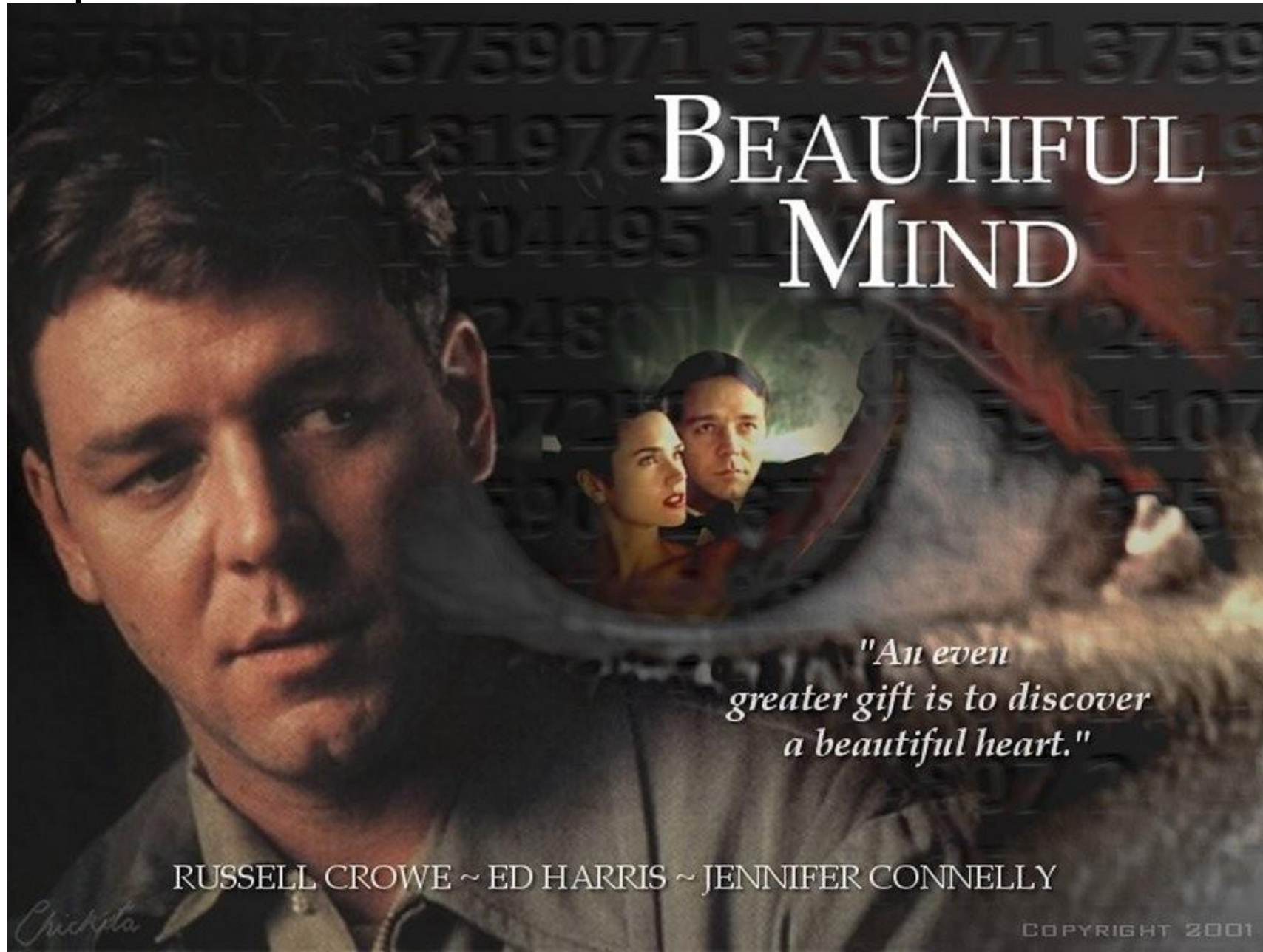


Juegos con adversarios

Búsqueda adversarial

- Examinar los problemas que surgen cuando tratamos de planificar acciones futuras en un mundo donde otros agentes planifican contra nosotros.
- Un buen ejemplo son los juegos de tableros.
- Los juegos con adversario, aunque bastante estudiados en IA, son una pequeña parte de **teoría de juegos** (muy usada en economía).

Una película del año 2001



John Nash

- Premio Nobel en Economía (1994)



Suposiciones típicas en IA

- La existencia de dos agentes cuyas acciones se alternan
- El valor de utilidad para cada agente es el opuesto al del otro agente
 - Crea la situación adversarial
- Ambientes totalmente observables
- En términos de teoría de juegos: Juegos de suma-cero de información perfecta
- Esas suposiciones se pueden relajar en etapas posteriores

Búsqueda versus juegos

- Búsqueda – sin adversario
 - Solución es un método (heurístico) para encontrar una meta
 - Las técnicas heurísticas pueden encontrar una solución *optimal*
 - Función de evaluación: estima el costo para ir desde una situación inicial a la meta a través de un nodo dado
 - Ejemplos: planificación de trayectoria, actividades de scheduling

Búsqueda versus juegos (Cont.)

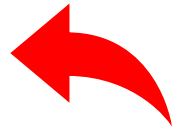
- Juegos – con adversario
 - La solución es una **estrategia** (la estrategia especifica movimientos para cada respuesta posible del oponente)
 - La optimalidad depende del oponente. ¿Por qué?
 - Los límites de tiempo fuerzan a una solución *aproximada*
 - La función de evaluación: evalúa la “bondad” de una posición en el juego
 - Ejemplos: ajedrez, damas, Othello, backgammon

Un tema reciente

- Inteligencia Artificial Generativa
 - ¿Hace cuánto tiempo se empezó a hablar de ello?

¿Qué es?

- Es la capacidad de crear contenido nuevo (original), a partir de un conjunto de instrucciones
 - Texto
 - Imágenes
 - Código
 - Audio
 - Video



Con creatividad

Impacto de la IAG

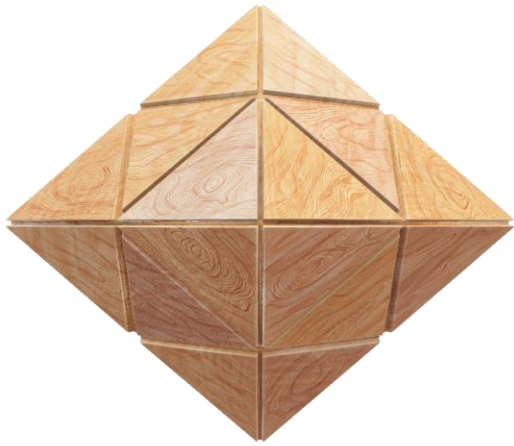
- Educacional
- Mercado laboral
- Incremento de la productividad

Cat and piano Picasso style

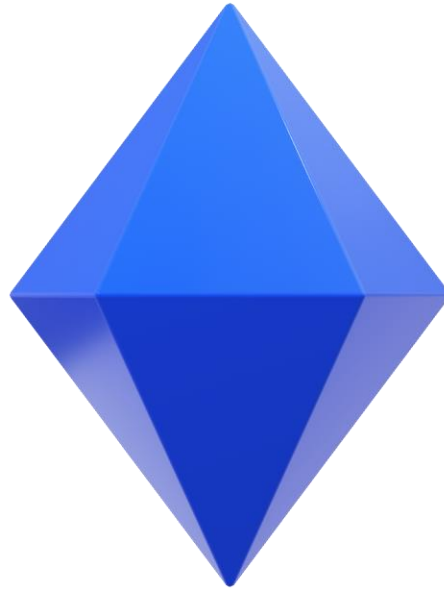


No es algo mágico

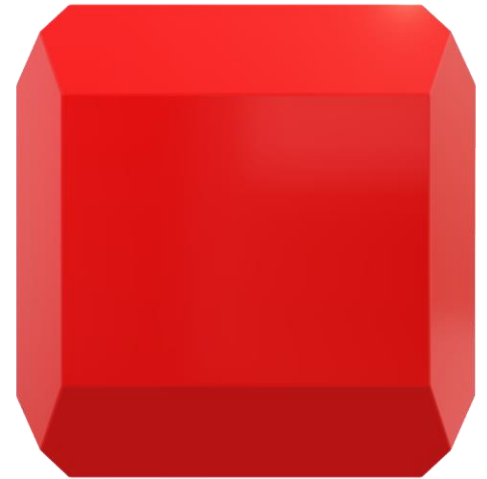
- Nosotros creamos esto



Datos



Algoritmos



Sistema “inteligente”



Algoritmo reconocimiento de imágenes



Sistema "inteligente"



Si



No

La esencia de la IA

- Netflix
- Spotify
- Comercio electrónico

GPT

- Significa **G**enerative **P**re-Trained **T**ransformer
- Desarrollado por la empresa OpenAI
- Orígenes a partir de investigaciones de redes neuronales y aprendizaje profundo, sobre todo de los modelos de lenguaje
- Versión Chat GPT-4 es lanzada en marzo 2023
- Chat GPT-5 es todavía misterioso..., emociones, mejor conexión con los humanos....



Próxima semana

- Búsqueda informada y/o con oponentes
- Procesos markovianos